사물인터넷 (Smart_IoT)
Sensors & Actuators

저작권

Copyright © 2015 Huins Corp. All Rights Reserved.

이 문서는 ㈜휴인스 Smart IoT 설계 시스템에 대한 정보 제공의 목적으로만 제공됩니다. ㈜휴인스는 문서에 수록된 정보의 완전성과 정확성을 검증하기 위해 노력하였으나, 발생할 수 있는 내용상의 오류나 누락에 대해서는 신속히 보완하여 알려드리겠습니다.

관련 URL 정보를 포함하여 이 문서에서 언급한 특정 소프트웨어 상품이나 제품은 해당 소유자가 속한 현지 및 국내외 관련법을 따르며, 해당 법률을 준수하지 않음으로 인해 발생하는 모든 결과에 대한 책임은 전적으로 사용자 자신에게 있습니다.

㈜휴인스에서 문서의 내용을 보완이 필요할 경우 예고 없이 변경할 수 있습니다. 자세한 내용은 아래 사이트를 참조하시기 바랍니다.

㈜휴인스 : http://www.huins.com

문서 정보

문서 개요

이 문서는 Smart IoT 설계 시스템을 사용하는 학습자에게 제공되는 문서입니다. 이 문서는 IoT 설계/실습 시스템을 사용하여 라즈베리파이를 배우고, IoT 제품을 설계 개발할 수 있는 능력을 키웁니다.

독자

이 문서는 IoT 설계/실습 시스템을 사용하는 학습자를 대상으로 합니다. 이 문서에서 IoT 에 관한 상세한 설명을 포함하고 있으며, 센서로부터 정보수집, 액츄에이터 제어, 안드로이드 앱 프로그램 등을 배울 수 있습니다. 프로그래밍 언어인 C, C++, Java, Python 에 대해서는 관련 서적을 참조하시기 바랍니다.

문의처

이 문서의 내용에 오류가 있거나 내용과 관련한 의문 사항이 있으면 아래의 연락처로 문의합니다.

연락처: 김미선 kms@huins.com 031-719-8200

문서 버전 및 이력

버전	일자	이력사항
1.0	2015. 04. 06	1.0 작성/배포
2.0	2016. 04. 29	2.0 수정 보완

표기 규칙

참고 표기



참고

독자가 참고해야 할 내용을 기술합니다.

주의 표기



주의

독자가 반드시 알아야 할 사항, 시스템 에러를 유발할 수 있는 사항, 수행하지 않았을 때 재산상의 피해를 줄 수 있는 사항을 기술합니다.

실습 시스템 이름, 구성품, 센서 이름, 액츄에이터 이름, 사용자 입력 값의 표기에 대해

이 문서에서 실습시스템, 구성품, 센서 이름, 액츄에이터 이름, 사용자 입력 값 등은 아래와 같이 표기합니다.

- 시스템 이름 : Smart IoT 실습시스템
- 구성품 이름 : Gateway, 센서, 액츄에이터, IoT IO Adapter, 케이블, SD 카드, 2Pin 연결선, 3Pin 연결선
- 센서, 액츄에이터 : 각각의 센서와 액츄에이터 이름.
- 사용자 입력 값: 실습 시에 사용자가 입력해야 하는 값을 말함.

소스 코드 표기

이 문서에서 소스 코드는 회색 바탕에 검정색 글씨로 표기합니다.

```
if(wiringPiSPISetup(SPI_CHANNEL, SPI_SPEED) == -1)
{
   fprintf(stdout, "wiringPiSPISetup Failed :%s\n", strerror(errno));
   return 1;
}

pinMode(CS_MCP3208, OUTPUT);
CODE Code = “We are Here, Our Planet”;
```

목차

Chapter 1 센서(Sensor) 8

1.1 온도 습도센서 11
1.2 조도센서 테스트 20
1.3 사운드 센서(음향센서) 30
1.4 인체 감지센서 38
1.5 초음파 센서 41
1.6 불꽃 센서(Flame sensor) 45
1.7 가스센서 49
1.8 인체감지센서 53

Chapter 2 액츄에이터(Actuator) 57

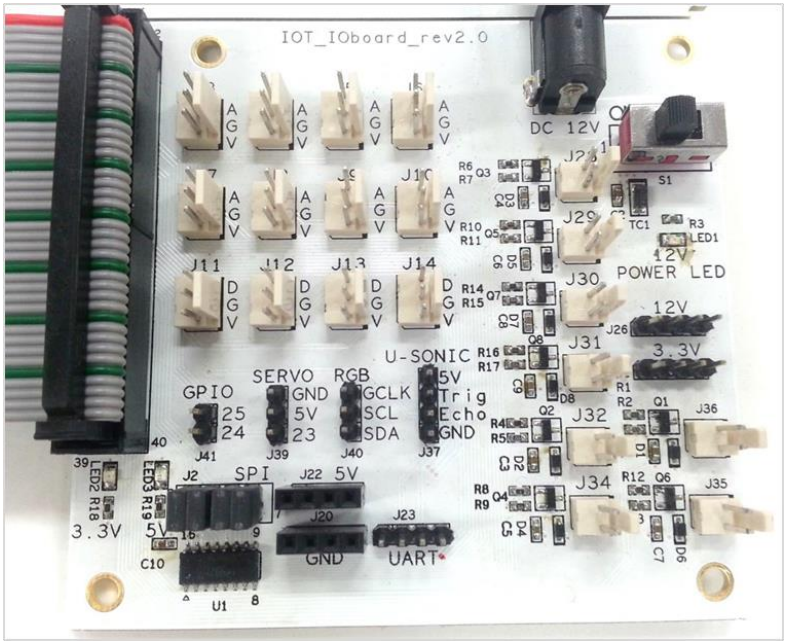
2.0 액츄에이터 실습 참고-sigint 이용 57
2.1 워터펌프 59
2.2 Fan 61
2.3 DC Motor 64
2.4 RGB Led 68
2.5 LED Bar 실습 73
2.6 Buzzer(부저) 실습 78
2.7 서보모터 실습 80

chapter 3 참고자료 84

Chapter 1 센서(Sensor)

센서로부터 센서값을 수집하거나 액츄에이터 제어하는 기능은 게이트 웨이에 연결된 IO Adapter 보드를 통해서 수행한다. IO Adapter 보드는 센서 입력 전원(DC 3.3V, DC 5V)과 출력(ananlog, digital)을 갖는 여러가지 센서들을 연결할 수 있도록 만든 회로 기판이다. 또한 IO Adapter 보드는 구동 전원(DC 5V, DC 12V)이 다른 여러가지 액츄에이터들을 연결하여 구동할 수 있도록 설계하였다.

모든 실습 시스템에 들어가는 IO Adapter 보드는 여기에 소개된 센서나 액츄에이터 뿐만 아니라 사용자가 개발하려고 하는 IoT 제품에 필요한 다른 센서와 액츄에이터를 확장할 수 있도록 도와준다. 설계 시스템 시험은 IoT Adapter 보드에 필요한 센서를 연결하고, 필요한 Actuator 를 구동하는 실습을 진행한다.



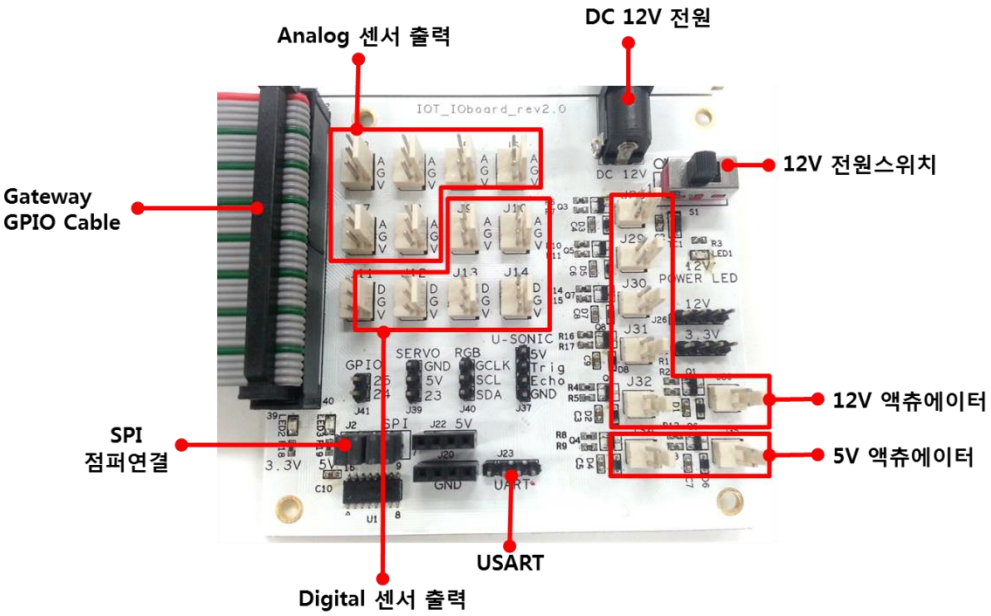
IO Adaptor 보드

IO Adapter 보드가 가지고 있는 커넥터들과 사양을 입출력을 구분하여 테이블로 나타내었다. 기본적으로 라즈베리 파이는 전원을 3.3V 사용한다. 만약 5V 를 라즈베리 파이에 공급하거나 GPIO 핀으로 입력이 될 경우 라즈베리 파손의 원인이 된다. 다양한 센서와 핀 맵에 적용하기 위해 설계된 IO 어댑터 보드는 실습에 아주 유용한 도구이다.

IO Adapter 보드 입출력 포트 구성

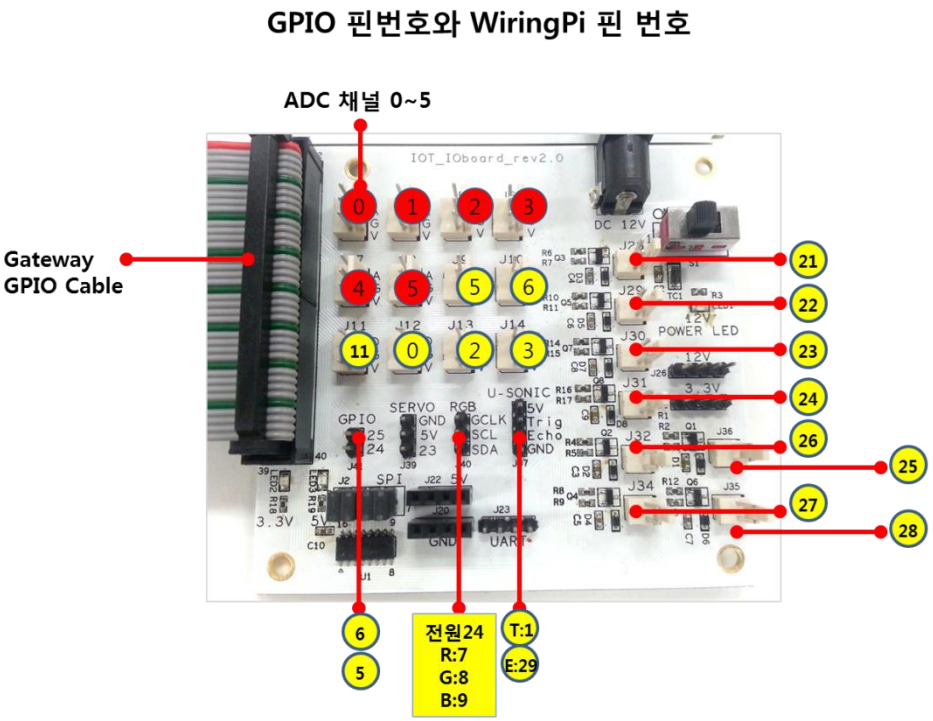
IO Adapter 보드의 입출력 사양		
항목	사양	줄력/입력

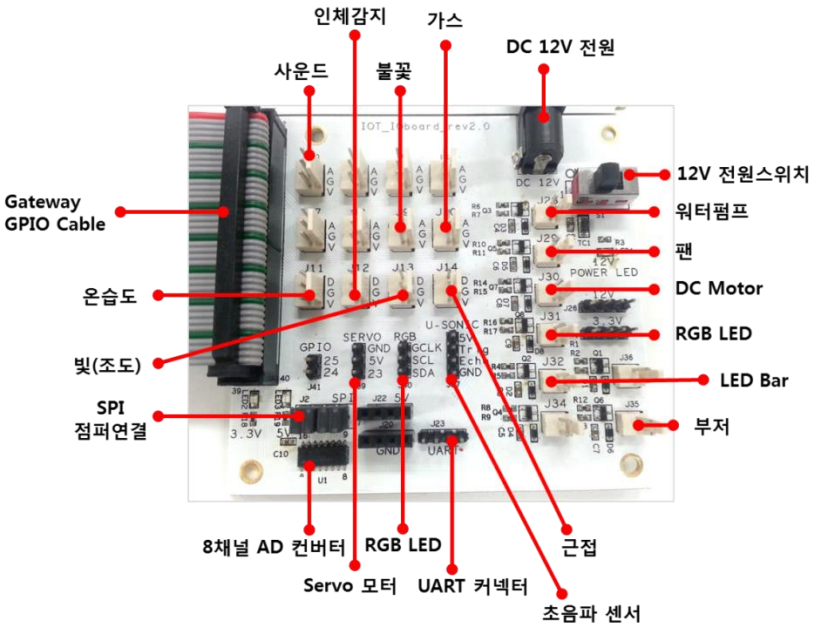
인터페이스	40pin GPIO(라즈베리 파이)	GPIO
전원 공급	DC 3.3V, DC 5V	GPIO
6채널 AD 컨버터	MCP3208, SPI 인터페이스	입력
DC 3.3V 디지털 센서 커넥터	6포트	입력
DC 12V 전원 출력	6포트	출력
DC 5V 전원 출력커넥터	2포트	출력
DC 3.3V UART 커넥터	1포트	출력
	1포트	출력
GPIO 출력	9 pin Header	출력



IO 어댑터 보드

각 실습과제에 따라 IO Adapter 보드에 센서와 액츄에이터를 연결하고, 라즈베리파이 프로그래밍을 한다. 센서에 필요한 전원과 출력 형태에 따라 적당한 커넥터에 연결을 해야한다. DC 12V 전원 입력은 대부분의 액츄에이터가 12V 전원을 쓰기 때문이다. 그러나, DC 5V 전원이 공급되는 액츄에이터를 위해 별로로 4 채널의 DC 5V 출력 커넥터가 존재한다. 아래 사진은 케이블로 연결한 IO Adapter 그림이다.



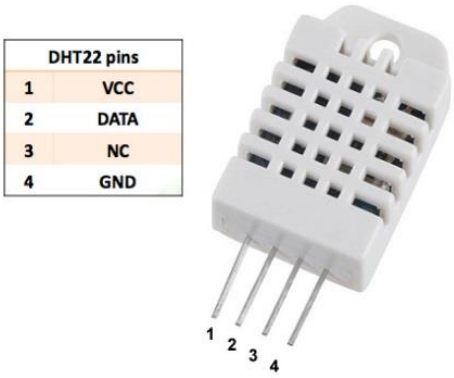


센서와 액츄에이터를 연결한 IO Adapter

Smart IoT 시스템의 센서는 모두 8 종을 지원한다. 센서들을 모두 같은 전원에 출력도 하나로 통일된 것들을 구매하기는 어렵다. 또한 보다 정확한 데이터를 수집해야 하는 상황에서라면 아날로그 출력의 센서값을 변환해서 사용하는 것이 정확성 측면에서 유리할 것이다. 여기서는 시스템에 사용되는 센서의 종류와 역할에 대해 자세한 명세서나 시험방법은 실습을 통해 하나씩 알아보기로 한다.

1.1 온도 습도센서

IoT 스마트 농장관리 시스템에 사용되는 센서를 알아보자. 실제 화분이나 식물농장의 상태에는 토양의 온습도를 측정하는 센서가 필수이지만 실습을 위해서 대기 온습도 센서를 사용하기로 한다.



DHT22/AM2302 온습도 센서

실습에서 사용되는 온습도 센서는 고정밀 디지털 온습도 센서 모듈 DHT22/AM2302 를 사용하기로 한다. 온도와 습도를 측정하고 이를 이용하여 제어하는 기술이 많은 분야에서 이용되고 있다. 아날로그 출력의 온습도 센서를 사용하여 마이크로컨트롤러에서 응용하기 위해서는 부가적인 회로나 AD 컨버터 회로가 별도로 필요한데 DHT22/AM2302 온습도 센서는 4.7KΩ~10KΩ 의 풀업 저항을 달아 1wire 통신을 이용해 센서 데이터를 획득할 수 있다.

요즘에는 신호 증폭, ADC, 보정, 통신 등의 회로를 내장하고 온도, 습도를 동시에 측정할 수 있는 센서들이 나오고 있다. DHT22/AM2302 은 DHT11 과 사용법이 비슷하나 DHT11 에 비해 정밀도가 높고 응답시간이 빠른 장점이 있다.

DHT22/AM2302 센서의 주요 특징을 아래 표로 나타내었다.

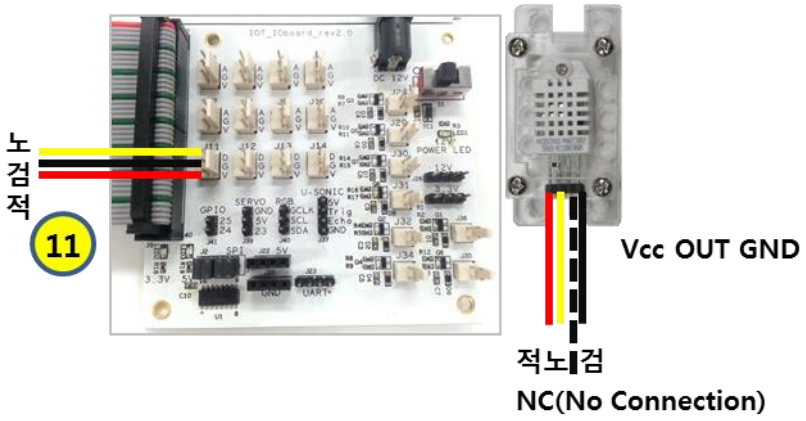
표 1 DHT22/AM2302 온습도 센서의 특징

1. 모델명 : AM2302	- 사용 전압 : 3.3-6V DC
2. 정밀도 및 해상도 : 0.1	- 싱글핀 디지털 데이터 출력
3. 습도 측정 범위 : 0-100%RH	- 측정 응답 시간 : min 2초
4. 온도 측정 범위 : -40~80℃	- 측정 조건 VDD = 5V , T = 25 ℃
5. 습도 측정 오차 : ±2%RH	- Pin Connection
6. 온도 측정 오차 : ±0.5℃	Pin 1 – 전원 VCC 3.5~5.5V DC
7. 저전력소모	Pin 2 - DATA (시리얼 양방향)
8. 부가회로가 불필요	Pin 3 – 사용안함(No Connection)

	Pin 4 - GND

DHT22/AM2302 온습도 센서의 통신방법, 프로토콜에 대한 상세한 내용은 데이터 북을 참조한다. (http://www.adafruit.com/datasheets/DHT22.pdf)

아래와 같이 연결한다.



IO 어댑터 보드와 습도센서의 연결

A. 라즈베리파이와 센서 연결 회로도

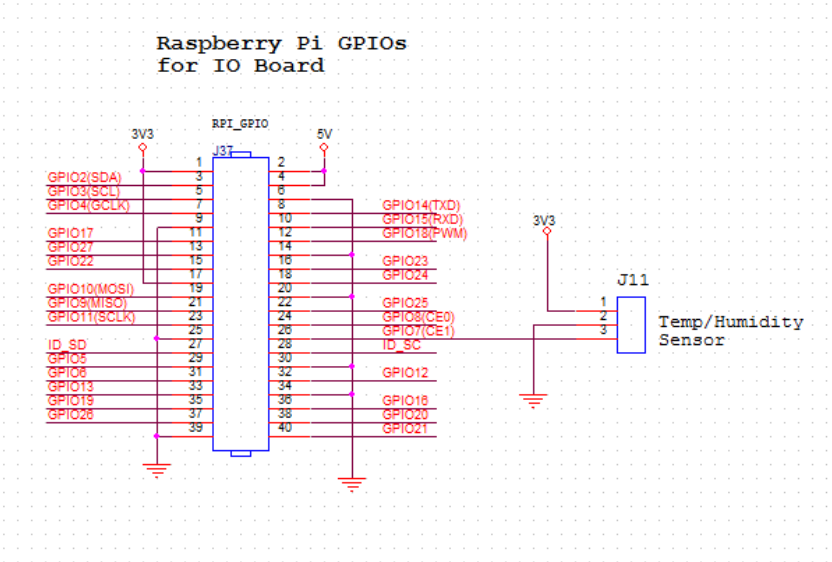


그림 1 라즈베리 파이와 습도센서 연결 회로도

B. wiringPi 설치

라즈베리 파이의 GPIO 를 제어할 수 있는 언어는 C, C++, C#, Python, Java 등 매우 다양하게 있다. 그 중에서 제일 많은 사용자들이 있고, 초기부터 최근까지 꾸준이 업데이트가 유지된 wiringPi 를 이용하기로 한다. (wiringPi – Gordon 이 개발, GNU LGPLv3 라이선스, 홈페이지 http://wiringpi.com)를 이용한다.

GPIO 란?

GPIO(General Purpose Input Output)란 하나의 연결 단자를 입력이나 출력으로 고정시키지 않고 선택적으로 사용할 수 있게 융통성을 높인 범용 입출력 단자를 말한다. GPIO 단자를 입력으로 프로그래밍하면 인터럽트 소스로 사용 가능하며, 마찬가지로 일부 GPIO 단자는 프로세서를 휴면 상태에서 깨우는 사건(event)를 생성하기도 한다.

1. 새로운 패키지를 실행하기 전에 꼭 해주어야 할것이 하나 있다. 바로 $sudo apt-get update 이다. 이 명령은 패키지 관리 서버로부터 이용 가능한 프로그램 패키지의 목록을 최신버전으로 업데이트 하는 명령이다.

```
$sudo apt-get update
```

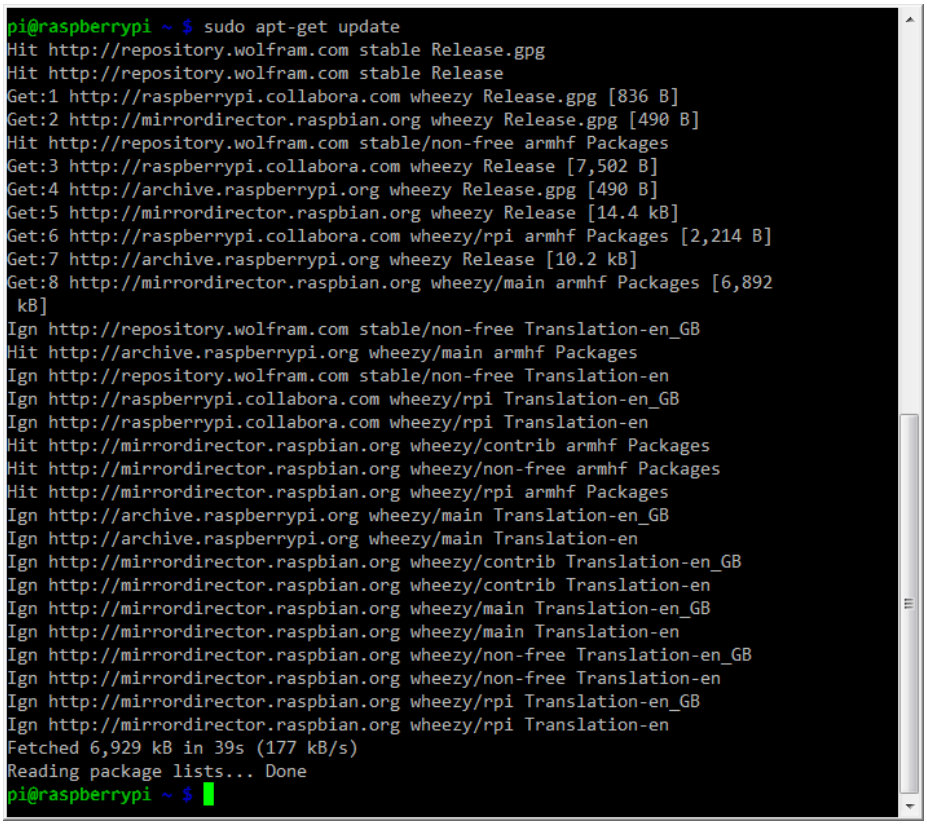


그림 2 sudo apt-get update 실행화면

2. 다음으로 업데이트 명령어로 갱신된 패키지 목록 중 컴퓨터에 설치된 프로그램이 업데이트 되었으면 새로운 버전으로 설치하게 해주는 작업이다. 사용하는 OS 배포버전에 따라 많은 업데이트가 이루어진 경우도 있으며 이러한 경우에는 시간이 오래 걸릴 수 도 있다.

```
$sudo apt-get upgrade
```

3. 소스관리 툴인 git 를 다운로드 한다.

```
$ sudo apt-get install git-core
```

4. git 를 이용하여 “wiringPi” 라이브러리를 다운로드하고 빌드한다.

```
$ git clone git://git.drogon.net/wiringPi
$ cd wiringPi
$ ./build
```

설치가 잘 되었는지 다음의 명령어로 확인한다.

```
$ gpio -v
```

```
pi@raspberrypi ~ $ gpio -v
gpio version: 2.21
Copyright (c) 2012-2014 Gordon Henderson
This is free software with ABSOLUTELY NO WARRANTY.
For details type: gpio -warranty

Raspberry Pi Details:
  Type: Model B+, Revision: 1.2, Memory: 512MB, Maker: Sony
pi@raspberrypi ~ $
```

그림 3 wiringPi 설치확인 명령 실행화면

```
$ gpio readall
```

명령어는 현재 GPIO 핀 상태를 모두 보여준다. 테이블 상단의 BCM 열의 번호가 회로도에 나왔는 GPIO 핀 번호를 나타내고, 옆의 wPi 열의 핀 번호가 wiringPi 라이브러리를 사용할 때의 실제 번호이다. 프로그램으로 구현시 우리는 wPi 의 핀 번호를 사용하게 된다.

```
pi@raspberrypi ~ $ gpio readall
 +-----+-----+---------+------+---+--B Plus--+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 |     |     |    3.3v |      |   |  1 || 2  |   |      | 5v      |     |     |
 |   2 |   8 |   SDA.1 | ALT0 | 1 |  3 || 4  |   |      | 5V      |     |     |
 |   3 |   9 |   SCL.1 | ALT0 | 1 |  5 || 6  |   |      | 0v      |     |     |
 |   4 |   7 | GPIO. 7 |   IN | 1 |  7 || 8  | 1 | ALT0 | TxD     | 15  | 14  |
 |     |     |      0v |      |   |  9 || 10 | 1 | ALT0 | RxD     | 16  | 15  |
 |  17 |   0 | GPIO. 0 |   IN | 0 | 11 || 12 | 0 | IN   | GPIO. 1 | 1   | 18  |
 |  27 |   2 | GPIO. 2 |   IN | 0 | 13 || 14 |   |      | 0v      |     |     |
 |  22 |   3 | GPIO. 3 |   IN | 0 | 15 || 16 | 0 | IN   | GPIO. 4 | 4   | 23  |
 |     |     |    3.3v |      |   | 17 || 18 | 0 | IN   | GPIO. 5 | 5   | 24  |
 |  10 |  12 |    MOSI | ALT0 | 0 | 19 || 20 |   |      | 0v      |     |     |
 |   9 |  13 |    MISO | ALT0 | 0 | 21 || 22 | 0 | IN   | GPIO. 6 | 6   | 25  |
 |  11 |  14 |    SCLK | ALT0 | 0 | 23 || 24 | 1 | ALT0 | CE0     | 10  | 8   |
 |     |     |      0v |      |   | 25 || 26 | 1 | IN   | CE1     | 11  | 7   |
 |   0 |  30 |   SDA.0 |   IN | 1 | 27 || 28 | 1 | IN   | SCL.0   | 31  | 1   |
 |   5 |  21 | GPIO.21 |  OUT | 0 | 29 || 30 |   |      | 0v      |     |     |
 |   6 |  22 | GPIO.22 |  OUT | 0 | 31 || 32 | 0 | IN   | GPIO.26 | 26  | 12  |
 |  13 |  23 | GPIO.23 |  OUT | 0 | 33 || 34 |   |      | 0v      |     |     |
 |  19 |  24 | GPIO.24 |  OUT | 0 | 35 || 36 | 0 | OUT  | GPIO.27 | 27  | 16  |
 |  26 |  25 | GPIO.25 |   IN | 0 | 37 || 38 | 0 | OUT  | GPIO.28 | 28  | 20  |
 |     |     |      0v |      |   | 39 || 40 | 0 | OUT  | GPIO.29 | 29  | 21  |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+--B Plus--+---+------+---------+-----+-----+
pi@raspberrypi ~ $
```

그림 4 GPIO 핀 상태확인 실행화면

dht22 온습도 센서는 1 wire 통신으로 온도데이터와 습도 데이터를 송신한다. 실습의 편의를 위해 우선 온도 데이터를 수집하고 다음으로 습도 데이터를 얻어오는 실습을 진행한다. 프로그램은 온도와 습도를 다 받게 되어 있으니 온 습도 데이터를 모두 보고 싶은 사용자는 소스코드를 수정하여 사용하면 된다.

다음은 온도 데이터를 읽어오는 소스이다. 소스코드를 살펴보자. (get_temperature.c **파일**)

```
/*
/*
 *      dht22.c:
 * Simple test program to test the wiringPi functions
 * Based on the existing dht11.c
 * Amended by technion@lolware.net
 */

#include <wiringPi.h>

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <sys/types.h>
#include <unistd.h>

//#include "locking.h"

#define MAXTIMINGS 85

int ret_humid, ret_temp;

static int DHTPIN = 7;  //

static int dht22_dat[5] = {0,0,0,0,0};

static uint8_t sizecvt(const int read)
{
  /* digitalRead() and friends from wiringpi are defined as returning a value
  < 256. However, they are returned as int() types. This is a safety function
*/

  if (read > 255 || read < 0)
  {
    printf("Invalid data from wiringPi library\n");
    exit(EXIT_FAILURE);
  }
  return (uint8_t)read;
}

int read_dht22_dat()
{
  uint8_t laststate = HIGH;
  uint8_t counter = 0;
  uint8_t j = 0, i;

  dht22_dat[0] = dht22_dat[1] = dht22_dat[2] = dht22_dat[3] = dht22_dat[4] =
0;
```

```
    // pull pin down for 18 milliseconds
    pinMode(DHTPIN, OUTPUT);
    digitalWrite(DHTPIN, HIGH);
    delay(10);
    digitalWrite(DHTPIN, LOW);
    delay(18);
    // then pull it up for 40 microseconds
    digitalWrite(DHTPIN, HIGH);
    delayMicroseconds(40);
    // prepare to read the pin
    pinMode(DHTPIN, INPUT);

    // detect change and read data
    for ( i=0; i< MAXTIMINGS; i++) {
      counter = 0;
      while (sizecvt(digitalRead(DHTPIN)) == laststate) {
        counter++;
        delayMicroseconds(1);
        if (counter == 255) {
          break;
        }
      }
      laststate = sizecvt(digitalRead(DHTPIN));

      if (counter == 255) break;

      // ignore first 3 transitions
      if ((i >= 4) && (i%2 == 0)) {
        // shove each bit into the storage bytes
        dht22_dat[j/8] <<= 1;
        if (counter > 16)
          dht22_dat[j/8] |= 1;
        j++;
      }
    }

    // check we read 40 bits (8bit x 5 ) + verify checksum in the last byte
    // print it out if data is good
    if ((j >= 40) &&
        (dht22_dat[4]  ==  ((dht22_dat[0]  +  dht22_dat[1]  +  dht22_dat[2]  +
dht22_dat[3]) & 0xFF)) ) {
          float t, h;

          h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
          h /= 10;
          t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
          t /= 10.0;
          if ((dht22_dat[2] & 0x80) != 0)  t *= -1;

            ret_humid = (int)h;
            ret_temp = (int)t;
            //printf("Humidity = %.2f %% Temperature = %.2f *C \n", h, t );
            //printf("Humidity = %d Temperature = %d\n", ret_humid, ret_temp);

      return ret_temp;
```

```
  }
  else
  {
    printf("Data not good, skip\n");
    return 0;
  }
}

int main (void)
{
  int received_temp;

  DHTPIN = 11;

  if (wiringPiSetup() == -1)
        exit(EXIT_FAILURE) ;

  if (setuid(getuid()) < 0)
  {
        perror("Dropping privileges failed\n");
        exit(EXIT_FAILURE);
  }
  while (read_dht22_dat() == 0)
  {
        delay(500); // wait 1sec to refresh
  }
  received_temp = ret_temp ;
  printf("Temperature = %d\n", received_temp);

  return 0;
}
```

편집을 완료하였다면 ctrl-x 를 눌르고, 같은 파일 이름이므로 y 를 누르고 엔터를 눌러 빠져나온다.

이제, gcc 명령을 이용하여 위 소스를 컴파일을 하고, 실행해보면 온도 값이 화면에 출력됨을 볼 수 있다.

static int DHTPIN = 7; 은 온 습도 센서 데이터의 출력 핀 번호를 지정한다. 1. 아래와 같이 컴파일을 수행한다.

```
$gcc -o get_temperature get_temperature.c -l wirlingPi
```

2. 아래 명령어로 온도 데이터를 확인하기 위해 실행한다.

```
pi@raspberrypi ~/smartfarm $ sudo ./get_temperature
Temperature = 26
```

아래의 소스는 습도 데이터를 읽어오는 소스코드이다. 온도와 동일한 구조와 방법으로 프로그램 되었다. (get_humidity.c **파일**)

```
/*
 *      dht22.c:
 *Simple test program to test the wiringPi functions
 *Based on the existing dht11.c
```

```
   * Amended by technion@lolware.net
   */

  #include <wiringPi.h>

  #include <stdio.h>
  #include <stdlib.h>
  #include <stdint.h>
  #include <sys/types.h>
  #include <unistd.h>

  //#include "locking.h"

  #define MAXTIMINGS 85

  int ret_humid, ret_temp;

  static int DHTPIN = 7;  //gpio 7

  static int dht22_dat[5] = {0,0,0,0,0};

  static uint8_t sizecvt(const int read)
  {
    /* digitalRead() and friends from wiringpi are defined as returning a value
    < 256. However, they are returned as int() types. This is a safety function
*/

    if (read > 255 || read < 0)
    {
      printf("Invalid data from wiringPi library\n");
      exit(EXIT_FAILURE);
    }
    return (uint8_t)read;
  }

  int read_dht22_dat()
  {
    uint8_t laststate = HIGH;
    uint8_t counter = 0;
    uint8_t j = 0, i;

    dht22_dat[0] = dht22_dat[1] = dht22_dat[2] = dht22_dat[3] = dht22_dat[4] =
0;

    // pull pin down for 18 milliseconds
    pinMode(DHTPIN, OUTPUT);
    digitalWrite(DHTPIN, HIGH);
    delay(10);
    digitalWrite(DHTPIN, LOW);
    delay(18);
    // then pull it up for 40 microseconds
    digitalWrite(DHTPIN, HIGH);
    delayMicroseconds(40);
    // prepare to read the pin
    pinMode(DHTPIN, INPUT);
```

```
    // detect change and read data
    for ( i=0; i< MAXTIMINGS; i++) {
      counter = 0;
      while (sizecvt(digitalRead(DHTPIN)) == laststate) {
        counter++;
        delayMicroseconds(1);
        if (counter == 255) {
          break;
        }
      }
      laststate = sizecvt(digitalRead(DHTPIN));

      if (counter == 255) break;

      // ignore first 3 transitions
      if ((i >= 4) && (i%2 == 0)) {
        // shove each bit into the storage bytes
        dht22_dat[j/8] <<= 1;
        if (counter > 16)
          dht22_dat[j/8] |= 1;
        j++;
      }
    }

    // check we read 40 bits (8bit x 5 ) + verify checksum in the last byte
    // print it out if data is good
    if ((j >= 40) &&
        (dht22_dat[4]  ==  ((dht22_dat[0]  +  dht22_dat[1]  +  dht22_dat[2]  +
dht22_dat[3]) & 0xFF)) ) {
          float t, h;

          h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
          h /= 10;
          t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
          t /= 10.0;
          if ((dht22_dat[2] & 0x80) != 0)  t *= -1;

            ret_humid = (int)h;
            ret_temp = (int)t;
            //printf("Humidity = %.2f %% Temperature = %.2f *C \n", h, t );
            //printf("Humidity = %d Temperature = %d\n", ret_humid, ret_temp);

      return ret_humid;
    }
    else
    {
      printf("Data not good, skip\n");
      return 0;
    }
  }

  int main (void)
  {
     int received_humid;

     DHTPIN = 11;
```

```
    if (wiringPiSetup() == -1)
            exit(EXIT_FAILURE) ;

    if (setuid(getuid()) < 0)
    {
            perror("Dropping privileges failed\n");
            exit(EXIT_FAILURE);
    }
    while (read_dht22_dat() == 0)
    {
            delay(500); // wait 1sec to refresh
    }
    received_humid = ret_humid;
    printf("Humidity = %d\n", received_humid);

    return 0;
}
```

편집을 완료하였다면 ctrl-x 를 눌르고, 같은 파일 이름이므로 y 를 누르고 엔터를 눌러 빠져나온다.

이제, gcc 명령을 이용하여 위 소스를 컴파일을 하고, 실행해보면 습도 값이 화면에 출력됨을 볼 수 있다.

static int DHTPIN = 7; 은 온습도 센서 데이터의 출력 핀 번호를 지정한다. 컴파일을 수행한다.

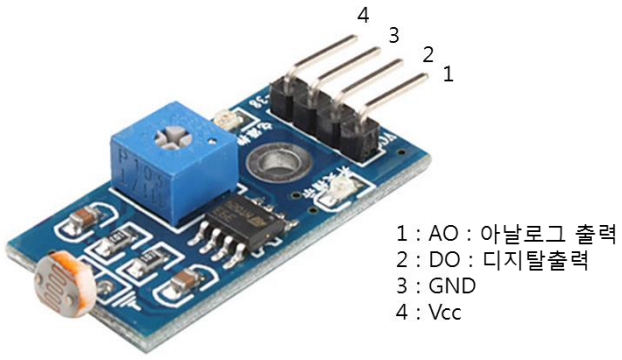
```
$gcc -o get_humidity get_humidity.c -l wirlingPi
```

아래 명령어로 온도 데이터를 확인하기 위해 실행한다.

```
pi@raspberrypi ~/smartfarm $ sudo ./ get_humidity
Humidity = 26
```

1.2 조도센서

조도센서는 광량(빛의 세기)를 센싱하여 전기적 신호로 출력하는 센서이다. 센서의 핀이 3 개로 구성되어 단순히 빛이 있다 없다의 출력을 내주는 디지털 조도센서가 있고, 아날로그 디지털 두 신호가 출력되는 4 핀의 조도센서가 있다. 여기서는 식물농장 시스템에 설치된 조도 센서가 디지털 아날로그 두가지 출력을 내는 4pin 조도센서를 싫습한다. 그리고 디지털 출력을 내는 3pin 조도센서를 테스트 한다.



4 핀 6495 조도센서

4 핀 조도센서

본 실습에서 사용되는 조도센서는 모델 6495 조도센서(4-핀)이다. 조도센서의 출력은 두가지를 이용할 수 있는데 일정 밝기 이상에서 하이가 출력되는 디지털 출력과 빛의 양에 따라 값이 아날로그로 변환되어 출력하는 아날로그 출력을 동시에 가지고 있다. 여기에서는 아날로그 출력을 이용한다. 아날로그 출력값은 8 채널 12bit 아날로그 디지털 컨버터인 MCP3208 을 이용하여 디지털 값을 출력하여 보기로 한다.

조도센서의 특징과 사양

특징	전기적 특성
모델명 : GL5537(5Pie Photo Resistor) 종류 : Cds Photo CELL 2. 감도 조절 (파란색 가변저항 조정)밝기와 주변 빛의 세기를 감지 3. 출력 : 아날로그 전압 출력, 디지털 스위치 출력 (0, 1) 4. Power dissipation : 100mW 5. Max Voltage : 150Vdc or AC 6. Temp.Range: : -30~+70℃	- 작동 전압 : 3.3V ~ 5V DC - 출력 : 아날로그 전압 출력, 디지털 스위치 출력 (0, 1) - Light Resistance at 10Lux(at 25): min 20, max 50Kohm- 측정 조건 VDD = 5V , T = 25 ℃ - Gamma Value at 10~100Lux: 0.7typ - Dark Resistance at 0 Lux(10sec after shut off 10Lux): min 2Mohm - Peak Spectrol Response: min 550, max 650nm

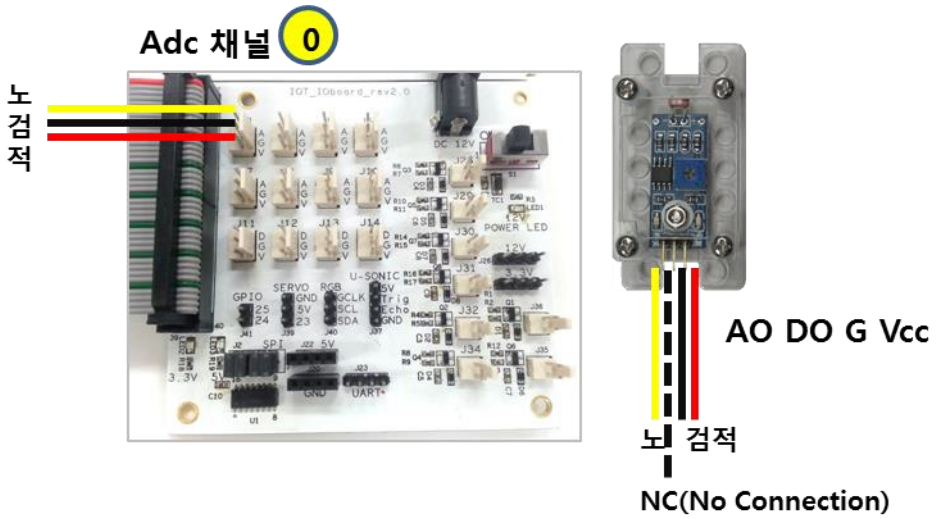


그림 5 4 핀 조도센서와 IO Adapter 보드 연결

ADC 컨버터 MCP3208 설명

MCP3208 은 8-Channel 12-Bit A/D Converters 이다. 즉, 센서의 출력값인 아날로그 값을 12 비트의 디지털 값으로 변환하는 ADC 칩이다. 예를들어 0v ~ 3.3v 의 센서 출력 값을 0 ~ 4095 (12bit) 의 디지털 값으로 변환해주는 기능을 해준다. 라즈베리파이는 범용적인 목적으로 입/출력을 담당하는 GPIO (General Purpose Input / Output) 를 가지고 있으며, SPI 통신, I2C 통신, UART 통신을 가지고 있지만, 아쉽게도 ADC 기능이 없어서 센서를 활용못하는 단점이 있다. 그래서, ADC 칩인 MCP3208 을 이용하여 외부 ADC 기능을 가지도록 한다. 이를 위해서는 라즈베리파이와 MCP3208 간에 SPI 통신을 이용하여 ADC 데이타를 받아와야 한다.

MCP3208 는 아래와 같은 핀 구조를 가지고 있다. 이를 라즈베리파이의 SPI 와 연결하기위해서는 아래의 회로도와 같이 연결해줘야 한다. MCP3208 은 총 8 개의 ADC 포트(CH0~CH7)를 가지고 있는데 이번 강좌에서는 CH0 에 가변저항을 달아 돌려가면서 ADC 값을 변경해보며 테스트를 해보도록 하겠다. 자신이 이용하고자하는 센서를 CH0 ~ CH7 에 연결하여 사용하여도 무방하다.

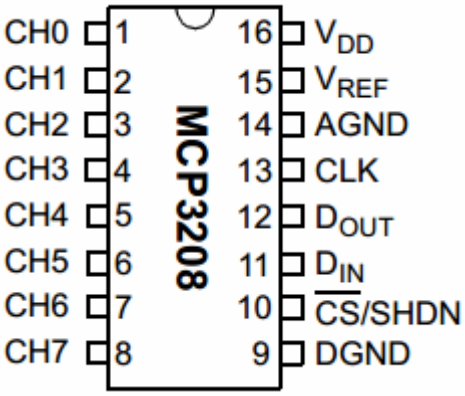


그림 6 MCP3208 Pin Map

Name	Function
V_{DD}	+2.7V to 5.5V Power Supply
DGND	Digital Ground
AGND	Analog Ground
CH0-CH7	Analog Inputs
CLK	Serial Clock
D_{IN}	Serial Data In
D_{OUT}	Serial Data Out
$\overline{CS}$/SHDN	Chip Select/Shutdown Input
V_{REF}	Reference Voltage Input

그림 7 2 MCP3208 Pin Description

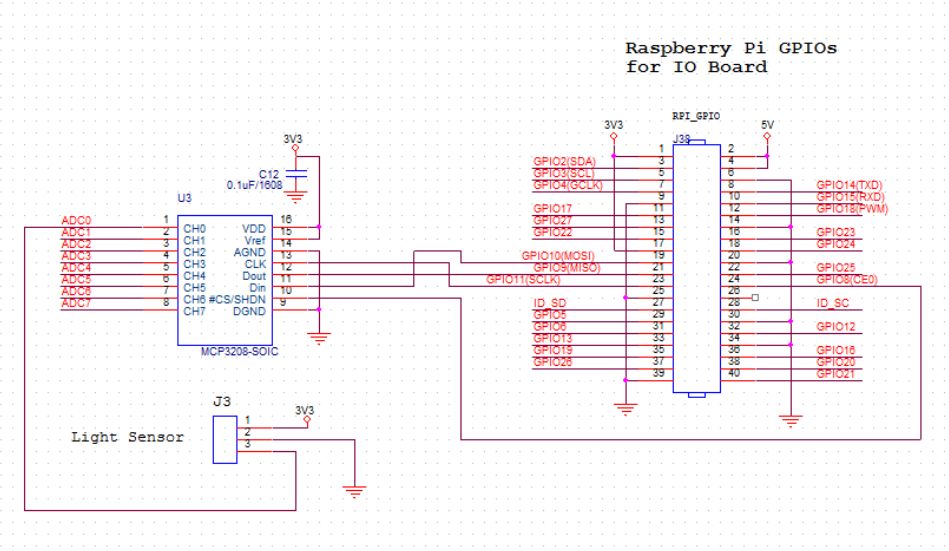


그림 8 라즈베리 파이와 ADC 간의 핀 연결도

SPI 테스트는 C 언어로 작성해 보고자 한다. 매우 간단하니 천천히 따라해 보자. 우선, 위와 같은 회로를 구성한 후 전원을 넣고, 라즈베리파이를 구동하자. 아래의 순서대로 소스코드를 작성하도록 하자.

1. **(SPI 및 I2C 활성화)** 라즈베리파이는 기본적으로 SPI 및 I2C 통신이 활성화 되어 있지 않다. 아래의 설정 파일을 수정하여 비활성화 설정된 두 기능 모두를 활성화 한다. 먼저 raspi-blacklist.conf 파일을 오픈한다, 파일에서는 #을 앞에 넣어주어 블랙리스트를 비활성화 시킨다.

```
$sudo nano /etc/modprobe.d/raspi-blacklist.conf
```

```
# blacklist spi and i2c by default (many users don't need them)

#blacklist spi-bcm2708
#blacklist i2c-bcm2708
blacklist snd-soc-pcm512x
```

```
blacklist snd-soc-wm8804
```

2. **(SPI 로드)** SPI 기능을 사용하기 위해서는 아래와 같이 커널로 부터 spi 드라이브를 로드해야한다.

```
gpio load spi
```

3. **(작업폴더 생성)** SPI 에 관한 설정이 끝났다. 다음은 자신의 작업 폴더 생성 및 이동하여 아래와 같이 작업 폴더를 설정해 주자.

```
mkdir smartfarm
cd smartfarm
```

4. **(소스코드 작성)**프로그램 작성 (편집기는 vi, nano, Leafpad 등 원하는 것을 사용한다.)

프로그램은 "Gordons Projects" 의 "wiringPi" 의 라이브러리를 이용할 것이다. 특히, 이번에 사용할 라이브러리는 wiringPiSPI 이다. 자세한 라이브러리를 확인하고자 하는 경우에는 wiringPiSPI 홈페이지를 참고한다.

위와 같이 내용을 전부 입력해주고, "spi-test.c" 라는 이름으로 저장해주자.

※ SPI 통신 프로그램은 마스터와 슬레이브로 나뉜다. 본 실습에서는 라즈베리파이가 마스터이고, MCP3208 이 슬레이브이다. 즉, 마스터인 라즈베리파이가 정해진 명령어를 슬레이브인 MCP3208 에게 정해진 명령을 날려주면 그에 해당되는 값을 전송해주는 시스템이다. 우리는 센서 값을 ADC 하여 라즈베리파이가 받을 필요가 있으므로 아래와 같은 규칙(MCP3208 데이타시트, 첨부파일 참조)으로 데이타를 전송해야 한다. 라즈베리파이는 총 3 개의 바이트를 보내주는데, 그 내용은 아래와 같다.

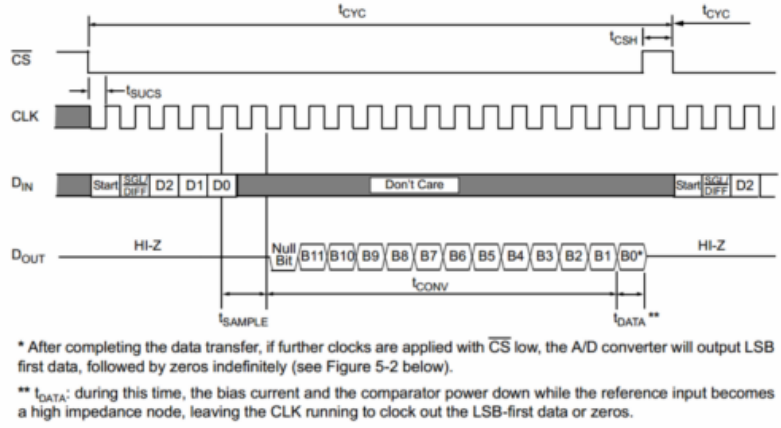
1 첫째 바이트 0 0 0 0 0 Start SGL D2

2 번째 바이트 D1 D0 x x x x x x

3 번째 바이트 x x x x x x x x

(D2,1,0 은 ADC 채널을 의미하고, x 는 어떠한 값도 상관없는 Don`t Care 를 의미한다.)

이와 같이 라즈베리파이는 3 바이트의 데이타를 보냄과 동시에 MCP3208 로부터 3 바이트의 데이타를 받게 되는데 그 데이타 중에서 밑으로부터 12 비트가 ADC 결과값이다. 이를 위의 소스와 같이 비트연산으로 하나의 데이타 값으로 변환하면 된다.



* After completing the data transfer, if further clocks are applied with CS low, the A/D converter will output LSB first data, followed by zeros indefinitely (see Figure 5-2 below).

** t_{DATA}: during this time, the bias current and the comparator power down while the reference input becomes a high impedance node, leaving the CLK running to clock out the LSB-first data or zeros.

그림 9 MCP3208 데이터 수신과정

Control Bit Selections				Input Configuration	Channel Selection
Single /Diff	D2	D1	D0		
1	0	0	0	single-ended	CH0
1	0	0	1	single-ended	CH1
1	0	1	0	single-ended	CH2
1	0	1	1	single-ended	CH3
1	1	0	0	single-ended	CH4
1	1	0	1	single-ended	CH5
1	1	1	0	single-ended	CH6
1	1	1	1	single-ended	CH7
0	0	0	0	differential	CH0 = IN+ CH1 = IN-
0	0	0	1	differential	CH0 = IN- CH1 = IN+
0	0	1	0	differential	CH2 = IN+ CH3 = IN-
0	0	1	1	differential	CH2 = IN- CH3 = IN+
0	1	0	0	differential	CH4 = IN+ CH5 = IN-
0	1	0	1	differential	CH4 = IN- CH5 = IN+
0	1	1	0	differential	CH6 = IN+ CH7 = IN-
0	1	1	1	differential	CH6 = IN- CH7 = IN+

그림 10 MCP3208 의 채널정보

다음은 조도 센서에서 데이터를 읽어오는 소스코드이다. 먼저 센서 데이터를 읽어오는 소스를 살펴보자. (getlightsensor .c 파일)

편집은 nano 편집기를 이용한다. 편집 명령은 $nano getlightsensor.c 명령으로 시작하고 편집이 완료되면 ctrl-x 를 누르고 파일네임 변경이 없을 경우 y 를 눌러 enter 로 빠져 나온다.

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <wiringPi.h>
#include <wiringPiSPI.h>

#define CS_MCP3208 8 //GPIO 8
#define SPI_CHANNEL 0
#define SPI_SPEED 1000000 //1Mhz

// spi communication with Rpi and get sensor data

int read_mcp3208_adc(unsigned char adcChannel)
```

```
 {
   unsigned char buff[3];
   int adcValue = 0;

   buff[0] = 0x06 | ((adcChannel & 0x07) >> 2);
   buff[1] = ((adcChannel & 0x07) << 6);
   buff[2] = 0x00;

   digitalWrite(CS_MCP3208, 0);
   wiringPiSPIDataRW(SPI_CHANNEL, buff, 3);

   buff[1] = 0x0f & buff[1];
   adcValue = (buff[1] << 8 ) | buff[2];

   digitalWrite(CS_MCP3208, 1);

   return adcValue;
 }

 int main(void) {

   unsigned char adcChannel_light = 0;

   int adcValue_light = 0;

   float vout_light;
   float vout_oftemp;
   float percentrh = 0;
   float supsiondo = 0;

   printf("start");


   if(wiringPiSetupGpio() == -1)
   {
          fprintf(stdout, "Unable to start wiringPi :%s\n", strerror(errno));
          return 1;
   }

   if(wiringPiSPISetup(SPI_CHANNEL, SPI_SPEED) == -1)
   {
          fprintf(stdout, "wiringPiSPISetup Failed :%s\n", strerror(errno));
          return 1;
   }

   pinMode(CS_MCP3208, OUTPUT);

   while(1)
   {
          adcValue_light = read_mcp3208_adc(adcChannel_light);

          //printf("Humiity   =   %u   temparature   =   %u\n",   adcValue_humi,
adcValue_temp);
          printf("light sensor = %u\n", adcValue_light);

          delay(100);
```

```
    }
    return 0;
}
```

편집이 완료되면 ctrl-x 를 누르고 파일네임 변경이 없을 경우 y 를 눌러 enter 로 빠져 나온다.

5. **(컴파일)** gcc 를 이용하여 소스 코드를 다음과 같이 컴파일 한다.

```
$gcc -o getlightsensor getlightsensor.c -lwiringPi
```

: 위의 컴파일 명령을 실행해 주면 'spi-test' 라는 실행 파일이 생성된다.

6. **(실행)** 프로그램을 실행한다

라즈베리파이의 GPIO 관련 프로그램 실행은 항상 루트 권한으로 실행해줘야 한다. 작성한 프로그램을 실행해 보자.

```
$sudo ./ getlightsensor
```

실행하게 되면 아래와 같이 터미널을 통해 ADC 된 조도센서의 데이터 값을 확인해 볼 수 있다. 이것은 한 채널의 ADC 값을 테스트해 본 것이고, 소스를 수정하여 각 채널의 ADC 를 확인해 볼 수도 있다.

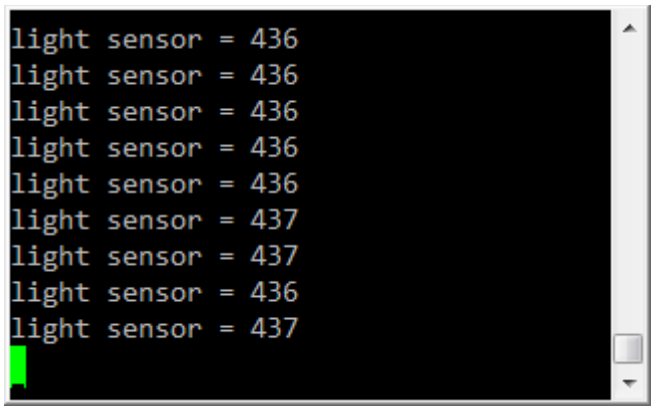


그림 11 아주 밝은 빛일때의 조도센서 출력 값

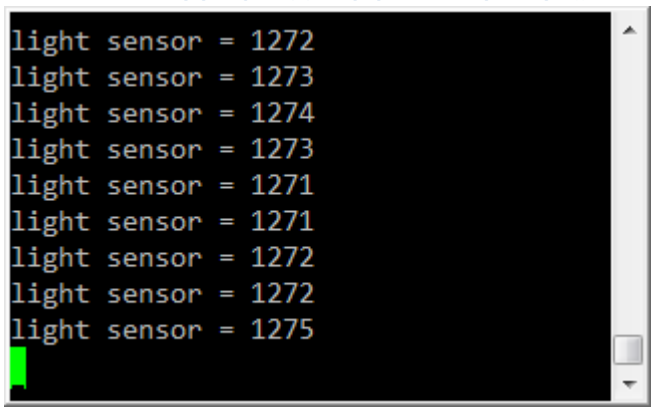


그림 12 보통 밝기에서의 조도센서 출력 값

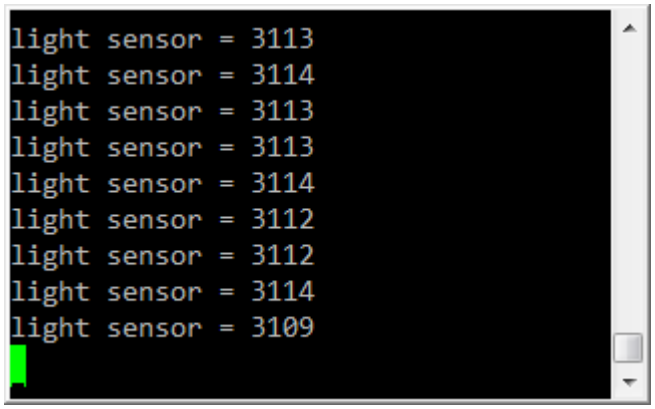
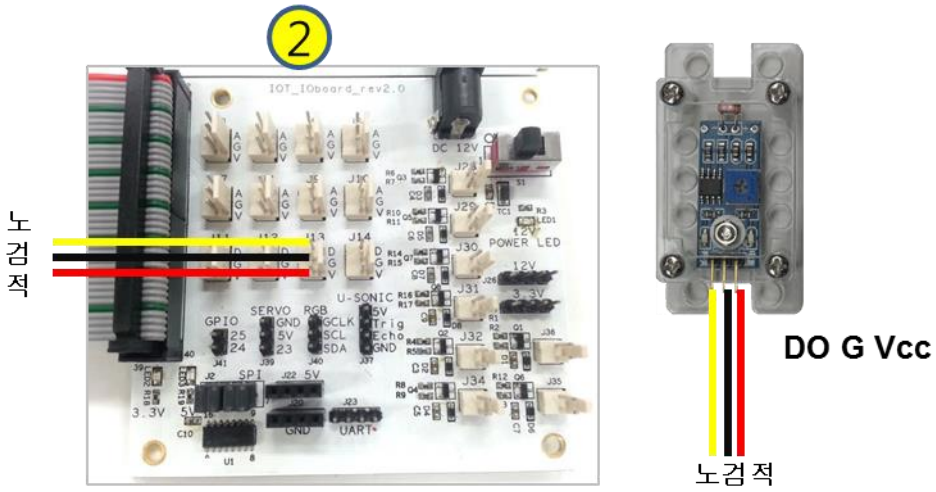


그림 13 어둡게 하였을 경우 조도센서 출력 값

3pin 조도센서

만약에 조도센서의 출력이 3 핀이라면 출력은 디지털 신호로 high(1), Low(0)을 출력하는 센서이다. 이러한 경우에는 앞에서 설명한 아날로그 값을 읽어서 판단하지 않고 가변저항을 돌려 정한 기준값과 비교하여 단순히 밝다, 어둡다 를 표시해 주어야 한다. 그 프로그램 예제를 살펴볼 것이다. 연결은 J13 에 한다.



3 핀 조도센서와 IO adapter 보드 연결

다음은 3-핀 조도센서에서 데이터를 읽어오는 소스코드이다. (lightdetect.c 파일)
$nano lightdetect.c 명령어로 소스를 입력한다.

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>
```

```
#define LIGHTSEN_OUT 2  //gpio27 - J13 connect

// ---------------------------------------------------------------------------
// main
int main(void)
{
  // sets up the wiringPi library
  if (wiringPiSetup () < 0)
  {
        fprintf  (stderr,  "Unable  to  setup  wiringPi:  %s\n",  strerror
(errno));
        return 1;
  }

  pinMode(LIGHTSEN_OUT, INPUT);

  // set Pin 17/0 generate an interrupt on high-to-low transitions
  // and attach myInterrupt() to the interrupt
  /*if ( wiringPiISR (LIGHTSEN_OUT, INT_EDGE_RISING, &myInterrupt) < 0 )
  {
        fprintf (stderr, "Unable to setup ISR: %s\n", strerror (errno));
        return 1;
  }*/

  // display counter value every second.
  while ( 1 )
  {
        //printf( "%d\n", eventCounter );
        //eventCounter = 0;

        if(digitalRead(LIGHTSEN_OUT) == 0)
              printf("light full ! \n");
        if(digitalRead(LIGHTSEN_OUT) == 1)
              printf("dark \n");

        delay( 200 ); // wait 1 second
  }

  return 0;
}
```

아래 명령으로 컴파일을 실행한다.

```
$gcc -o lightdetect lightdetect.c -lwiringPi
```

아래 명령으로 실행한다.

```
$sudo ./lightdetect
pi@raspberrypi ~/smartfarmtest $ sudo ./lightdetect

light full !
light full !
light full !
light full !
dark
dark
dark
```

```
dark
dark
light full !
light full !
```

1.3 사운드 센서(음향센서)

사운드 센서란 주변에서 흔히 볼 수 있는 마이크를 생각하면 된다. 소리가 발생할 경우 소리 센서는 소리의 크기와 지속시간에 따른 아날로그 신호를 출력한다. 소리로 전등을 켜고 끄거나 또는 주차 중인 차 안에 설치하여 주변에 소리가 발생할 경우 카메라가 동작하는 기능들은 사운드 센서를 사용해서 구현할 수 있는 기능들이다. 주변에서 여러가지 응용범위가 많은 사운드 센서를 실습하기로 한다. [그림 8]은 사운드 센서의 모양과 핀 맵을 보여준다.

표 1 사운드 센서 사양과 특징

사양	특징
크기	32mm*17mm*15mm(length*width*height)
Chip	LM393, 전기 콘덴서 마이크
작동 전압	DC 3-6v
출력	아날로그/디지털
LED	Power LED, Sensor Indicator LED

사운드 센서도 마찬가지로 출력은 4 핀으로 되어 있다. 전원과 GND 과 센서의 출력을 Analog 로 출력하는 Aout 과 HIGH, LOW 로 출력하는 Digital 출력핀이 그것이다. 아래는 사운드 센서의 외관과 핀맵을 나타낸다.



Pin1 – Analog Out
Pin2 – GND
Pin3 – VCC전원
Pin4 – Digital Out

그림 14 소리센서(음향센서의 외형과 핀 맵

여기서 사운드 센서 테스트는 아날로그 입력을 받아 아날로그 값을 출력하는 것을 테스트 한다. 스마트 홈 보드의 U4 인 MCP3208 IC 는 SPI 인터페이스를 사용한다. 이종 컴퓨터간의 데이타 통신을 위해 사용되는 것으로 센서, ADC, 다른 MCU 와의 통신에 많이 사용되며, 마스터 및 슬레이브로 나누어 상호간의 통신을 할 수 있다. 라즈베리파이와 8-Channel 12-Bit A/D

Converters 인 MCP3208 을 SPI 기능을 통하여 통신하여, MCP3208 를 이용하여 센서 값을 받아오는 테스트를 해보도록 하겠다. 아래는 대표적인 SPI 인터페이스회로이다.

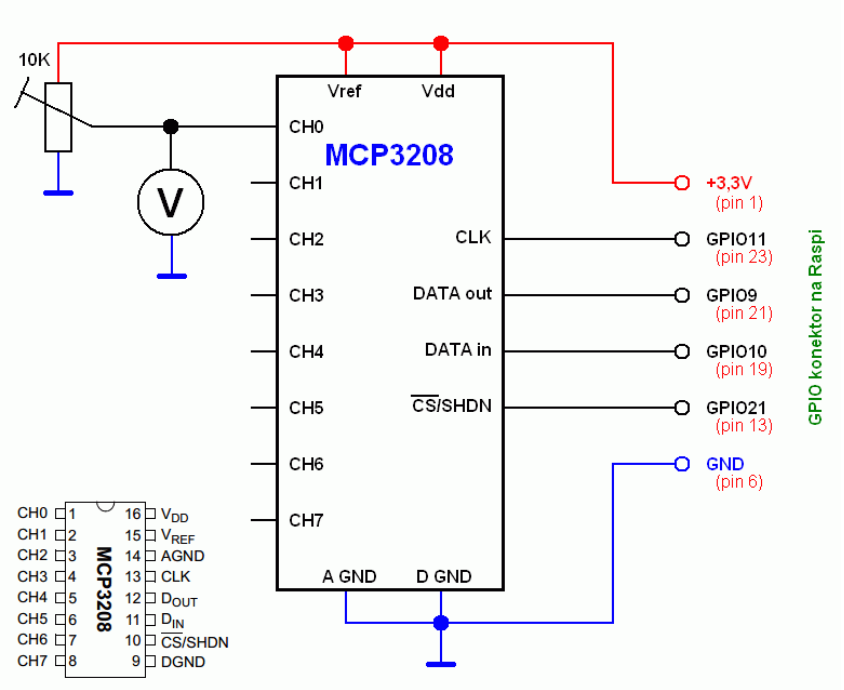


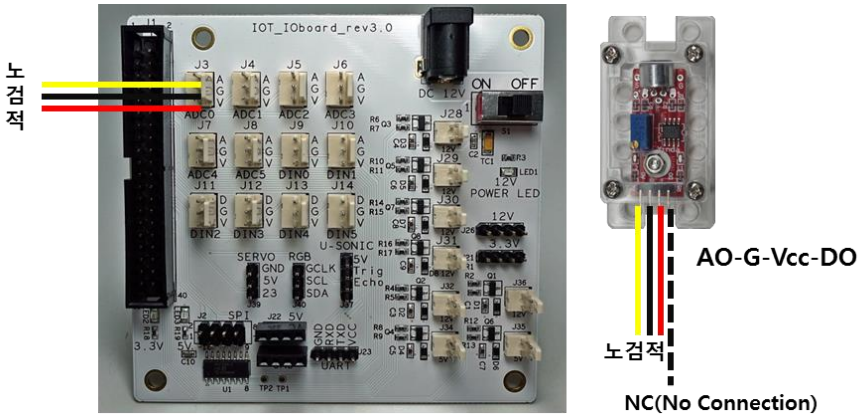
그림 15 아날로그 센서의 ADC 연결도

위와 같이 연결되어 SPI 인터페이스로 사운드 센서의 데이터를 획득할 수 있다. 아날로그 데이터를 얻기 위해 J15 에 연결된 모양을 아래 그림에 나타내었다.



그림 16 사운드 센서 연결도

시스템에서 IO Adapter 보드와 연결된 그림을 보자. 그림에는 디지털 출력과 아날로그 출력이 있는 4 핀 사운드 센서 연결도를 보여준다. 사운드 센서가 디지털 출력만을 지원하는 3 핀 센서라면 J11 ~ J14 커넥터에 연결하면 된다.



사운드 센서와 IO 어댑터 보드의 연결

센서 캘리브레이션 (Calibration, 보정)

다른 센서와 마찬가지로 사운드 센서도 보정을 거친 후 사용해야 한다. 방법은 전원을 연결하면 센서모듈 중앙의 두 개의 LED 중 한 개 전원 LED 에 불이 들어온다. 이때 다른 LED 도 켜져 있다면 항상 신호가 들어오는 것이므로 가변저항을 돌려 약간만 시계, 반시계 방향으로 돌려도 신호 출력 LED 가 꺼지는 상태까지 보정해 주면 된다.

소스코드를 입력하자 (sounddetect.c) 소스를 편집하는 명령은 nano 명령어를 사용한다. $nano sounddetect.c 이다.

```
$nano home-sound.c

#include <stdio.h>
#include <string.h>
#include <errno.h>

#include <wiringPi.h>
#include <wiringPiSPI.h>

#define CS_MCP3208 8 //GPIO 8

#define SPI_CHANNEL 0
#define SPI_SPEED 1000000 //1Mhz

// spi communication with Rpi and get sensor data

int read_mcp3208_adc(unsigned char adcChannel)
{
   unsigned char buff[3];
   int adcValue = 0;

   buff[0] = 0x06 | ((adcChannel & 0x07) >> 2);
   buff[1] = ((adcChannel & 0x07) << 6);
   buff[2] = 0x00;

   digitalWrite(CS_MCP3208, 0);
   wiringPiSPIDataRW(SPI_CHANNEL, buff, 3);
```

```
    buff[1] = 0x0f & buff[1];
    adcValue = (buff[1] << 8 ) | buff[2];

    digitalWrite(CS_MCP3208, 1);

    return adcValue;
}

int main(void) {

    unsigned char adcChannel_sound = 2;

    int adcValue_sound = 0;       //0번 채널 - 사운드 센서

    printf("start");

    if(wiringPiSetupGpio() == -1)
    {
            fprintf(stdout, "Unable to start wiringPi :%s\n", strerror(errno));
            return 1;
    }

    if(wiringPiSPISetup(SPI_CHANNEL, SPI_SPEED) == -1)
    {
            fprintf(stdout, "wiringPiSPISetup Failed :%s\n", strerror(errno));
            return 1;
    }

    pinMode(CS_MCP3208, OUTPUT);

    while(1)
    {
            adcValue_sound = read_mcp3208_adc(adcChannel_sound);

            printf("sound = %u\n", adcValue_sound);

            delay(100);
    }
    return 0;
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

```
$gcc -o sounddetect sounddetect.c -lwiringPi
```

아래 명령어로 실행하면 아날로그 사운드 데이터가 출력되는 것을 볼 수 있다.

```
$ sudo ./sounddetect

pi@raspberrypi ~/smartsafe$ sudo ./sounddetect

startsound = 2126
sound = 2125
sound = 2126
```

```
sound = 2128
sound = 2127
sound = 2126
sound = 2128
sound = 2127
sound = 2128
sound = 2128
sound = 2127
```

모든 센서들의 출력은 아날로그 출력을 기본적으로 가지고 있다. 정밀한 센서값과 필요할 때 언제든지 센서값을 읽어 처리가 가능하다는 면에서 아날로그 출력을 사용하는 장점이 있다. 사용상의 어려움으로 디지털 출력을 선호하고, 디지털 출력을 혼합하여 나오는 센서들이 대부분이다. 아날로그 출력을 얻기 위해서는 위의 방법을 사용한다.

디지털 출력을 사용하는 사운드 센서 실습 1 – 소리가 있나 없나 ?

사운드 센서의 디지털 출력을 이용하는 실습을 진행한다. 앞서 아날로그 출력을 이용하여 실습을 하였는데 아날로그 출력을 가지고 각종 소리의 유무, 소리 크기등을 판단하는 것이 약간은 어려울 수 있겠다고 생각되어 좀 더 쉬운 코드를 제공한다. 아래는 디지털 출력이 있는 사운드 센서 연결도를 나타내었다.

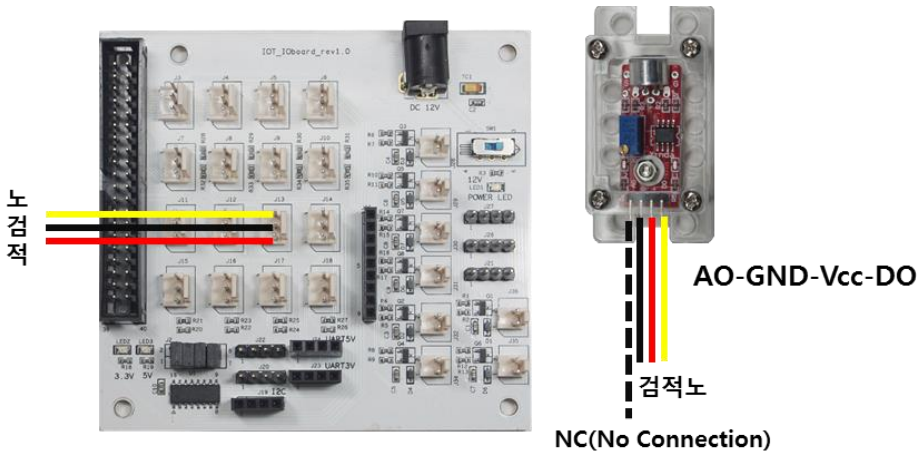


그림 17 디지털 출력이 있는 사운드 센서 연결

소스를 보면 인터럽트 방식으로 SOUND_IN 2 //gpio27 에 할당된 핀이 올라오는 순간에 인터럽트 처리 함수에서 soundFlag = 1; 로 하여 사운드가 있고 없고를 나타낸다. 소스코드를 입력한다. (soundtest.c)

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>

// Use GPIO Pin 7, which is Pin 11 for wiringPi library

#define SOUND_IN 2  //gpio27
```

```
  unsigned char soundFlag = 0;
  // ------------------------------------------------------------------------
  // myInterrupt:  called every time an event occurs
  void myInterrupt(void) {
    soundFlag = 1;

  }

  // ------------------------------------------------------------------------
  // main
  int main(void) {
    // sets up the wiringPi library
    if (wiringPiSetup () < 0) {
            fprintf  (stderr,  "Unable  to  setup  wiringPi:  %s\n",  strerror
(errno));
            return 1;
    }

    // and attach myInterrupt() to the interrupt
    if ( wiringPiISR (SOUND_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
            fprintf (stderr, "Unable to setup ISR: %s\n", strerror (errno));
            return 1;
    }

    while(1)
    {
            if(soundFlag == 1){
                    printf( "SoundDetect \n");
                    soundFlag = 0;
            }
            else
            {
                    printf("No Sound \n");
                    soundFlag = 0;
            }
            delay(100);
    }
    return 0;
  }
```

핀을 정의하는 부분

```
  #define SOUND_IN 2  //gpio27
```

인터럽트 루틴

```
  // myInterrupt:  called every time an event occurs
  void myInterrupt(void) {
    soundFlag = 1;
  }
```

인터럽트 함수 선언과 인터럽트를 받는 방식을 설정

```
  if ( wiringPiISR (SOUND_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
            fprintf (stderr, "Unable to setup ISR: %s\n", strerror (errno));
            return 1;
    }
```

무한 루프문

```
while(1)
  {
}
```

에 주의하면서 코딩을 한다.

컴파일 한다. $ gcc -o soundtest soundtest.c -lwiringPi

실행한다. $ sudo ./soundtest

결과를 확인하면 사운드 유무를 100msec 마다 알려준다.

```
$ sudo ./soundtest
No Sound
No Sound
SoundDetect
SoundDetect
```

디지털 출력을 사용하는 사운드 센서 실습 2 – 소리가 큰가 작은가 ?

본 실습에서는 약 0.5 초 동안 펄스(사운드 발생시 증가)를 세어 카운트 숫자가 크면 큰소리가 장기간 발생되고, 펄스 가운터 숫자가 작으면 작은소리가 짧게 발생했다는 것을 알 수 있도록 프로그램 하는 실습을 한다. 연결도는 디지털 사운드 출력을 이용하는 것과 동일하게 연결한다. 소스코드를 나타내었다. 인터럽트 루틴에서 카운터 숫자를 증가시키는 것과 0.5 초마다 사운드 센서 카운터를 디스플레이 하는 것을 반복하는 코드로 구현된다. 소스코드를 입력한다. (soundtest2.c)

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>


// Use GPIO Pin 7, which is Pin 11 for wiringPi library

#define SOUND_IN 2  //gpio27

// the event counter - not use
volatile int eventCounter = 0;
unsigned char soundFlag = 0;
// ---------------------------------------------------------------------------
// myInterrupt:  called every time an event occurs
void myInterrupt(void) {
   eventCounter++;

}

// ---------------------------------------------------------------------------
// main
int main(void) {
  // sets up the wiringPi library
```

```
  if (wiringPiSetup () < 0) {
      fprintf (stderr, "Unable to setup wiringPi: %s\n", strerror (errno));
      return 1;
  }

  // set Pin 17/0 generate an interrupt on high-to-low transitions
  // and attach myInterrupt() to the interrupt
  if ( wiringPiISR (SOUND_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
      fprintf (stderr, "Unable to setup ISR: %s\n", strerror (errno));
      return 1;
  }

  // display counter value every second.
  while ( 1 ) {
    printf( "%d\n", eventCounter );
    eventCounter = 0;
    delay( 500 ); // wait 1 second
  }

  return 0;
}
```

소스코드를 입력하였다면 컴파일 한다.

$gcc -o soundtest2 soundtest2.c -lwiringPi

실행한다. $sudo ./soundtest2

결과를 보면 큰소리 일때와 작은소리 일때의 카운터 숫자가 다르게 나타나는 것을 확인할 수 있다. 소리의 유무와 소리 크기에 따라 제어를 하는 방법은 여러 분야에 응용할 수 있으므로 잘 알아두도록 한다.

```
$sudo ./soundtest2
0
142
215
146
201
0
206
0
105
141
199
268
129
0
0
0
0
126
150
```

1.4 인체 감지센서

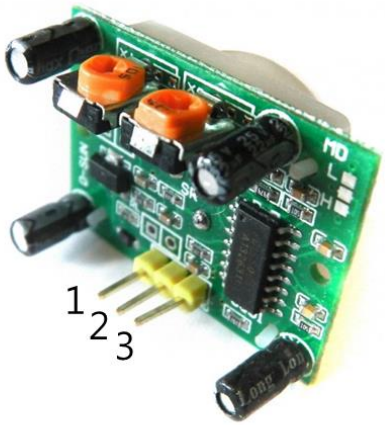
인체감지센서는 Pyroelectric ("Passive") InfraRed sensors 라고도 하며 일정한 적외선을 띤 물체가 움직이는 것을 감지하는 센서이며 주로 인체감지에 쓰인다. 적외선을 띤 물체가 감지되어 일정한 양의 적외선을 감지하게 되면, 출력을 내 보내고 적외선을 띤 물체가 없을 때는 출력을 내보내지 않는다. 돔 형태의 창(window)는 외부의 환경으로부터 센서 내부의 회로 및 감지부를 보호하고 원하는 파장대의 적외선만 통과시키는 광학 필터의 기능을 한다. 보통은 사람을 감지하여 일정시간 불을 켜는 용도로 사용한다.

표 2 인체감지 센서의 사양

사용 전류	50μA
사용전압	3V ~ 5V
동작 온도 범위	0℃ ~ + 70℃
센서 미 감지시 출력(HIGH)	4V
센서 감지시 출력(LOW)	0.4V
감지 각도	110 Degree
감지 거리	7M
Size	28mm×36mm
무게	25g

돔 형태의 창(window)는 외부의 환경으로부터 센서 내부의 회로 및 감지부를 보호하고 원하는 파장대의 적외선만 통과시키는 광학 필터의 기능을 한다. 보통은 사람을 감지하여 일정시간 불을 켜는 용도로 사용한다.

아래는 인체감지 센서의 외관과 Pin 기능을 보여준다.



Pin1 – VCC 전원
Pin2 – Digital Out
Pin3 – GND

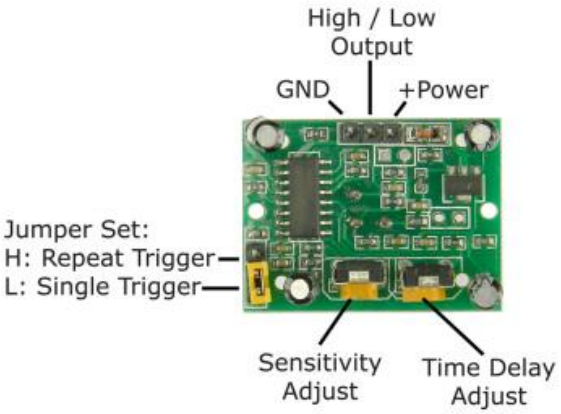
그림 18 인체감지 센서의 외형과 핀 맵

앞장에 설명한 사운드 센서와 거의 같은 방법으로 인체감지 센서를 테스트 한다. 전등에 사용되는 인체감지 센서는 감지가 된 후 약 5 초 정도 전등을 키고 OFF 하는 방식인데 여기에서 사용한 인체 감지 센서도 같은 방식으로 동작한다. 그 외 필요한 설정은 아래 설정을 보고 시험해야 한다.

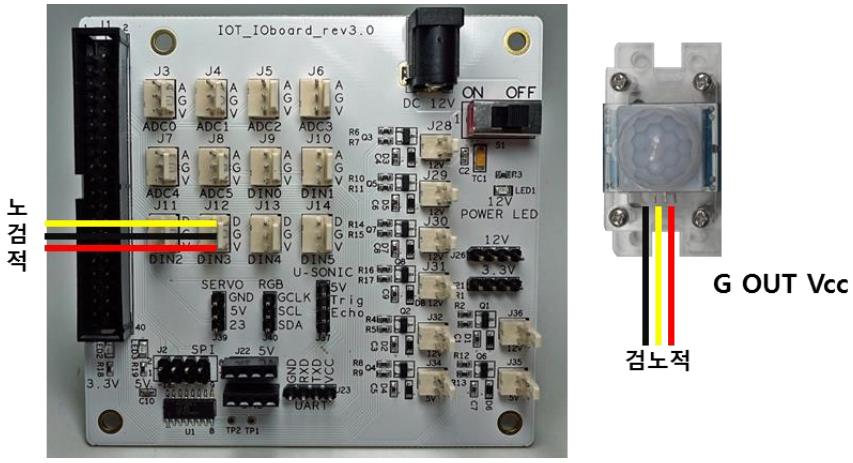
Calibration- 센서보정

감지 센서를 사용하기 전에 두 가지 컬리브레이션을 진행해야 한다. 아래[그림 8]을 보면 Sensitivity Adjust 와 Time Delay Adjust 가 있다. Sensitivity Adjust 는 인체감지 센서의 민감도를 조절하는 가변저항이고, Time Delay Adjust 인체가 감지되어 출력핀이 하이가 유지되는 시간을 조절하는 가변저항이다.

Sensitivity Adjust 가변저항을 왼쪽 끝까지 돌리면 민감도가 가장 약하고, 오른쪽으로 돌리면 민감도가 증가한다. 실습시에는 **왼쪽 끝**으로 돌려서 민감도를 제일 약하게 하는 것이 편할 것이다. Time Delay Adjust 가변저항은 오른 쪽으로 돌리면 하이가 출력되는 시간이 길어지고, 왼쪽으로 돌리면 약 4 초 후에 로우로 떨어지게 된다. 실습에서는 **가장 왼쪽으로** 돌려 일찍 로우상태(대기상태)로 되도록 하는 것이 결과를 확인하기 쉽다.



인체감지 센서 연결을 그림처럼 연결한다.



인체 감지센서 연결도

센서의 자세한 스펙은 아래 링크에서 다운로드 한다.

(http://www.mpja.com/download/31227sc.pdf)에서 데이터 쉬트를 다운 받을 수 있다.

인체감지 센서를 시험하는 소스코드를 아래에 나타낸다. (motiondetect.c)

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>


// Use GPIO Pin 6, which is Pin 22 for wiringPi library
#define MOTION_IN 0  //gpio17


// the event counter
volatile int eventCounter = 0;
unsigned char flag_detect;
// ------------------------------------------------------------------------
// myInterrupt:  called every time an event occurs
void myInterrupt(void) {
   eventCounter++;
   flag_detect = 0x01;
}


// ------------------------------------------------------------------------
// main
int main(void) {
  // sets up the wiringPi library
  if (wiringPiSetup () < 0) {
      fprintf (stderr, "Unable to setup wiringPi: %s\n", strerror (errno));
      return 1;
  }

  // set Pin 17/0 generate an interrupt on high-to-low transitions
  // and attach myInterrupt() to the interrupt
  if ( wiringPiISR (MOTION_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
```

```
        fprintf (stderr, "Unable to setup ISR: %s\n", strerror (errno));
        return 1;
    }

    // display counter value every second.
    while ( 1 )
    {
      if(flag_detect == 0x01)
    {
            printf("Motion Detect\n");
            flag_detect = 0;
    }
      //eventCounter = 0;
      delay( 100 ); // wait 1 second
    }

    return 0;
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

$gcc -o motiondetect motiondetect.c -lwiringPi

컴파일하여 실행하면 Motion Detec 메시지를 볼 수 있다. 인체를 감지하면 약 3~4 초 동안 같은 상태를 유지하고 초기화 되는 식으로 동적하기 때문에 인지가 되어 있는 동안에는 다시 인지를 하지 못한다.

실행 명령어는 $ sudo ./motiondetect 이고, 디텍트가 되면 하이 펄스를 감지하여 보여주는 화면이다.

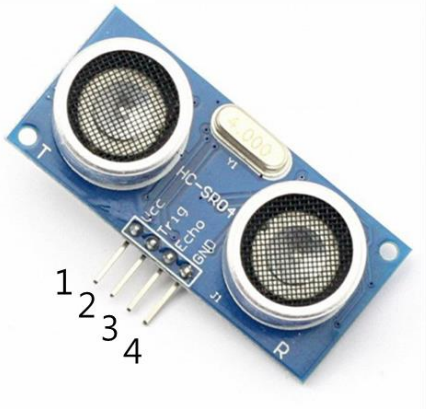
```
pi@raspberrypi ~/smartguard $ sudo ./motiondetect
 No detect
 No detect
 No detect
 No detect
Detect 1
high
high
high
high
```

1.5 초음파 센서

초음파 센서는 40KHz 의 펄스 신호를 출력하여 반사되어 돌아오는 펄스의 시간을 측정하여 거리를 계산한다. 여기서는 HC-SR04 초음파 센서를 사용한다. 초음파 센서의 자세한 스펙은 데이터 쉬트를 참고한다.

초음파 센서 데이터북 http://www.micropik.com/PDF/HCSR04.pdf

보통 초음파센서에는 4 개의 신호선이 존재하는데 각 각 VCC, Triger, Echo, GND 신호가 된다. VCC 와 GND 는 5v 라인에 연결하고, Triger 핀과 Echo 핀은 라즈베리파이의 GPIO 핀에 연결해준다.아래는 센서의 외관과 핀 맵을 보여준다.



Pin1 – VCC전원
Pin2 – Trigger 신호
Pin3 – Echo 입력신호
Pin4 - GND

그림 19 HC-SR04 초음파 센서의 외형과 핀 맵

HC-SR04 의 전기적 특성을 나타냈는데, 초음파 센서의 입력 전원으로 +5V 를 공급하는것에 주의한다. 출력은 펄스를 받아 계산하므로 센서의 Trigger 신호는 GPIO 핀으로 받게 된다.

표 3 HC-SR04 초음파 센서의 전기적 사양

Working Voltage	DC 5 V
Working Current	15mA
Working Frequency	40Hz
Max Range	4m
Min Range	2cm
MeasuringAngle	15 degree
Trigger Input Signal	적어도 10uS 이상의 TTL pulse
Echo Output Signal	Input TTL lever signal and the range in proportion
Dimension	45*20*15mm

초음파 센서의 타이밍도를 나타낸다.

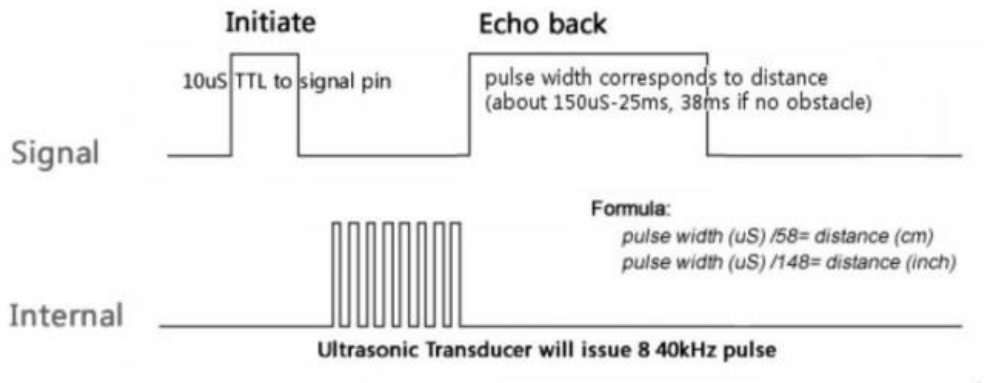
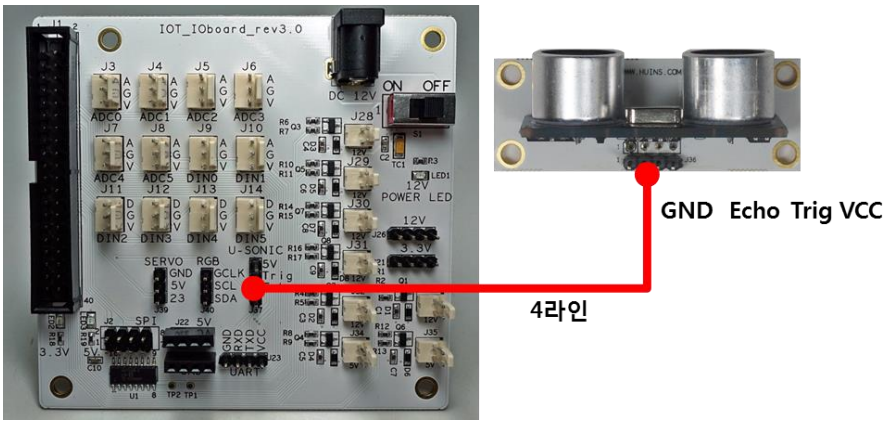


그림 20 초음파 센서의 타이밍도

위 그림은 센서의 작동원리를 설명해주는 그림이다. 우선 triger 핀에 10us 정도의 High 신호를 주면 초음파센서는 40khz 펄스를 자동적으로 8 번을 발생시키고 펄스를 발생시킨 직후에 echo 핀은 high 되고 반사된 초음파가 감지 되었을때 echo 핀이 low 가 된다. 이 Echo 핀이 high 였다가 low 가 되는데 걸리는 시간을 측정하여서 그 시간을 초음파의 속도/2(즉 58)로 나누면 거리가 나온다.

초음파 센서를 실습하기 위해 IO adapter 보드의 J16 에 VCC, GND, Echo 신호선을 연결하고 Trigger 신호는 J25 의 5 번 핀에 연결한다.



초음파 센서와 IO 어댑터 보드 연결도

nano 편집기를 이용하여 소스,파일을 생성한다.

```
$nano ultrasonic.c
```

소스파일 편집기에서 아래 코드를 입력한다. (ultrasonic.c)

```
#include<stdio.h>
#include<wiringPi.h>

#define trigPin 25      //gpio 26, pin 5 of J25
#define echoPin 4       //gpio  J16 - pin3, GPIO 23

int main(void)
{
  int distance=0;
```

```
    int pulse = 0;

    long startTime;
    long travelTime;

    if(wiringPiSetup () == -1)
    {
            printf("Unable GPIO Setup");
            return 1;
    }

    pinMode (trigPin, OUTPUT);
    pinMode (echoPin, INPUT);

    for(;;)
    {
            digitalWrite (trigPin, LOW);
            usleep(2);
            digitalWrite (trigPin, HIGH);
            usleep(20);
            digitalWrite (trigPin, LOW);

            while(digitalRead(echoPin) == LOW);

            startTime = micros();

            while(digitalRead(echoPin) == HIGH);
            travelTime = micros() - startTime;

            distance = travelTime / 58;

            printf( "Distance: %dcm\n", distance);
            delay(200);
    }
}
```

입력을 마치고 ctrl-x 를 입력하면 파일을 저장할것이냐고 묻는데 이때 y 를 입력한다.

Y 를 입력하고 파일이름을 묻어오면, 이름을 변경하지 않을 경우에는 그대로 Enter 를 치면 빠져 나오게 된다.

약 100usec 의 트리거 신호를 주고, 초음파 센서에서 측정한 거리값을 초 단위로 나타낸다. 이것을 0.2 초(200msec) 에 한번씩 출력하도록 프로그램 하였다.

입력을 다 했다면 컴파일을 수행한다.

```
$gcc -o ultrasonic ultrasonic.c -lwiringPi
```

아래명령으로 실행을 시키고 결과를 확인한다. 프로그램 실행을 중단할때는 ctrl – c 키를 누른다. 손이나 책을 센서 앞에 대면 거리가 변하는 것을 볼 수 있다.

```
$sudo ./ultrasonic
pi@raspberrypi ~/safeguard $ sudo ./ultrasonic
Distance: 268cm
Distance: 268cm
Distance: 266cm
Distance: 267cm
```

```
Distance: 269cm
Distance: 269cm
Distance: 268cm
Distance: 266cm
Distance: 266cm
Distance: 10cm
Distance: 7cm
Distance: 8cm
Distance: 8cm
Distance: 11cm
Distance: 293cm
Distance: 31cm
Distance: 266cm
Distance: 267cm
Distance: 268cm
```

1.6 불꽃 센서(Flame sensor)



화재, 가스 주의

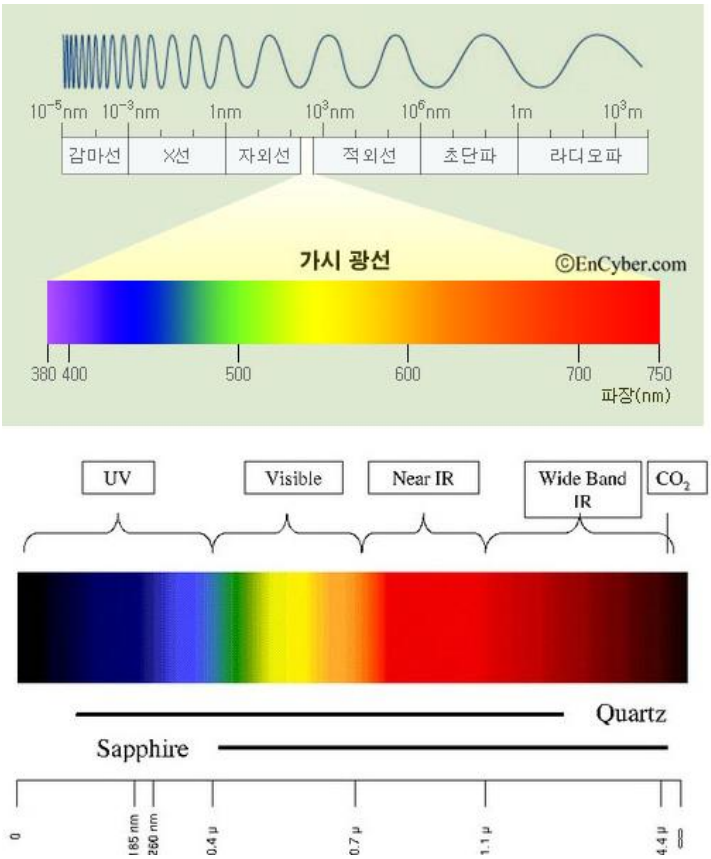
불꽃센서와 가스센서를 비롯하여 화재경보 시스템 시험시 간단하게 이용할 수 있는 장비로 라이터, 부탄가스, 가스레인지 등으로 화재나 신체상의 위험을 주의해야 한다.



참고

불꽃 센서는 760 ~ 1100nm 사이의 파장 광원을 감지하여 출력하므로 야외나 햇빛이 직접 드는 곳에서는 오작동하거나 센싱 결과를 신뢰할 수 없다.

불꽃 감지 센서는 적외선 LED 를 통해 불꽃에서 감지되는 적외선 파장을 감지하여 이를 아날로그 혹은 디지털 신호로 변환해 주는 센서이다.



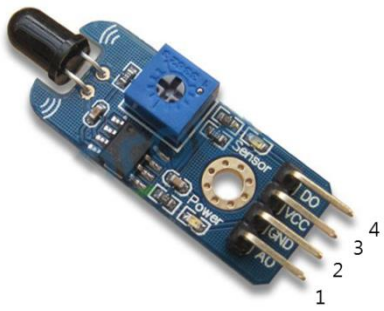
이미지출처 : 가시광선의 범위 - encyber.com

불꽃의 파장(주파수) 760nm ~ 1100nm 대역을 감지하는 센서로 소방활동 로봇 제작 및 화재 진압에 중요한 화재 경보기 제작에 활용이 가능하며 불꽃센서의 동작 환경은 25~85 도(섭씨)에서 작동 되며 불꽃을 감지하는 거리에 따라 아날로그 값이나 불꽃의 존재 유무를 알려주는 디지털 값으로 출력된다.

불꽃 센서의 제품 특징

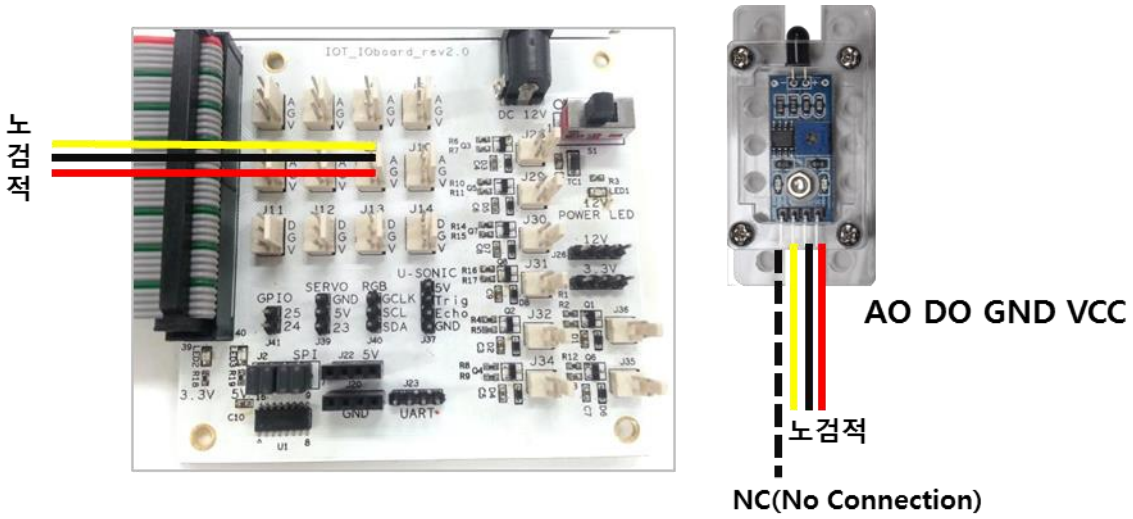
동작전압	3.3V to 5V
감지 범위	20cm (4.8V) ~ 100cm (1V)
감지 대역(파장)	760nm ~ 1100nm
응답 시간 :	15usec
인터페이스 :	아날로그/디지탈
크기 :	22x30mm

불꽃 센서의 이미지와 출력 Pin Map 을 나타내었다.



1 AO : 아날로그 출력
2 GND
3 VCC 전원
4 DO 디지털 출력

불꽃 센서 외관과 핀맵



IO 보드와 불꽃 센서 연결도

불꽃 센서로부터 출력을 얻는 소스코드는 아래와 같다. 불꽃 센서의 디지털 출력으로부터 불꽃이 있을 때와 없을 때의 출력을 얻는데 불꽃이 없을 때 HIGH 신호에서 qnfRhcdmf 감지하면 LOW 신호로 내려가는 시점을 인지해 불꽃이 있음을 알려준다. 인터럽트 방식을 사용해 FALLING EDGE 에서 신호를 발생하여 등록한 이벤트 함수를 실행시킨다.

```
if ( wiringPiISR (SOUND_IN, INT_EDGE_FALLING, &myInterrupt) < 0 ) {
      fprintf (stderr, "Unable to setup ISR: %s\n", strerror (errno));
      return 1;
}
```

nano flaedetect.c 로 소스파일을 작성한다.

```
nano flamedetect.c


#include <stdio.h>
#include <string.h>
```

```
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>


// Use GPIO Pin 7, which is Pin 11 for wiringPi library

#define SOUND_IN 11  //gpio7


// the event counter
volatile int eventCounter = 0;

// ------------------------------------------------------------------------
// myInterrupt:  called every time an event occurs
void myInterrupt(void) {
   eventCounter++;

}


// ------------------------------------------------------------------------
// main
int main(void) {
  // sets up the wiringPi library
  if (wiringPiSetup () < 0) {
      fprintf (stderr, "Unable to setup wiringPi: %s\n", strerror (errno));
      return 1;
  }

  // set Pin 17/0 generate an interrupt on high-to-low transitions
  // and attach myInterrupt() to the interrupt
  if ( wiringPiISR (SOUND_IN, INT_EDGE_FALLING, &myInterrupt) < 0 ) {
      fprintf (stderr, "Unable to setup ISR: %s\n", strerror (errno));
      return 1;
  }

  // display counter value every second.
  while ( 1 ) {
    printf( "%d\n", eventCounter );
    //eventCounter = 0;
    delay( 1000 ); // wait 1 second
  }

  return 0;
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

컴파일한다.

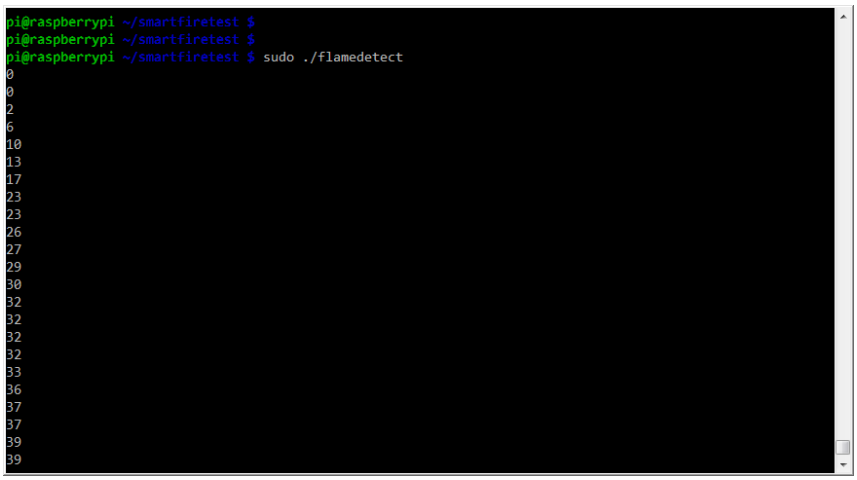
```
$gcc -o flamedetect flamedetect.c -lwiringPi
```

아래의 명령어로 실행한다.

```
$sudo ./flamedetect
```

거리에 따라 가스라이터의 반응을 조절하기 위해 센서 위 가변저항을 조절한다. 아날로그 출력을 입력으로 할 경우 불꽃의 유무뿐만 아니라 불꽃까지의 거리를 측정할 수 있지만 약 100cm 내의 거리에서 측정하므로 크게 의미는 없다. 아날로그 값을 읽어 출력을 센싱하는 방법은 현재 연결되어 있는 커넥터를 ADC 입력으로 연결해서 프로그램 하면 된다..

간단한 방법으로 가스라이터를 켜보면 켜보면 불꽃 센서가 반응하여 카운터 값이 증가되는 것을 볼 수 있다.



1.7 가스센서

주의

라이터, 부탄가스, 가스레인지 등으로 시험시 화재나 신체상의 위험을 주의해야 한다.

MQ 시리즈 가스 센서는 종류마다 다른 가스를 센싱하기도 한다. 시험에서는 MQ135 를 사용하여 LPG 가스나 부탄등을 검출하였는데 정상적으로 작동하였다 스펙에 적힌 용도의 센서를 쓰면 가장 좋지만 만약 없다면 다른 종류의 가스를 써서 시험을 진행한다.

MQ2 : Combustible Gas, Smoke
MQ5 : LPG, Natural Gas, Town Gas
MQ9 : Carbon Monoxide, Coal Gas, Liguefied Gas
MQ135 : Ammonia, Aromatics, Sulfer, Benzene vapor, other harmful

MQ135 는 내부에 포함된 히터와 센서로 맘모니아, 생활가스, 오염가스등을 검출해낼 수 있는 모듈이다. 내부에 포함된 히터가 적정온도로 올라가서 안정적인 결과를 얻기 위해서는 대기시간(burn-in time)이 필요하다. 때문에 MQ-2, MQ-3, MQ-4, MQ-5, MQ-6, MQ-7 모듈 종류별로 사용법이 약간씩 틀립니다. 센서가 보드에 결합되어 있는 경우 potentiometer 를 이용해서 히터에 걸리는 전압을 조절해서 센서의 감도를 조절한다.(센서만 있는 경우는 저항이나 PWM 핀으로 조절) 특정 가스에 대한 감도를 높이기 위해서는 다양하게 테스트 해 볼 필요가

있다. MQ135 의 경우에는 빠른 response time 을 갖고 있어서 거의 즉시 값을 추출할 수 있다. 아날로그 출력신호는 0~1023 사이의 값을 출력하므로 0~100%로 적절히 변환해서 사용할 수 있다.

반도체식 가스센서는 세라믹 반도체 표면에 가스가 접촉했을 때 일어나는 전기전도도의 변화를 이용하는 것이 많으며 대부분 대기 중에서 가열하여 사용되는 일이 많아 고온에서 안정한 금속산화물(세라믹스)이 주로 사용된다 금속 산화물은 반도체의 성질을 나타내는 것이 많고 이중 금속원자가 과잉(산소결핍)인 경우에는 n 형 반도체, 금속원자가 결핍인 경우에는 p 형 반도체가 사용된다. 이러한 세라믹반도체 중 전기전도도가 크고 융점이 높아서 사용온도 영역에서 열적으로 안정한 성질을 가진 반도체가 센서에 이용하고 있다.

반도체 가스센서는 1)대부분 유독가스 가연성가스에 어떤 응답을 나타내어 감지할 수 있는 가스의 종류가 많고, 2)센서제작이 용이하고 검출회로의 구성이 간단하다는 특징이 있다 그러나 감지하려는 가스만을 감지할 수 있는 선택성이 우수한 가스센서는 적고 아직도 연구개발 중에 있다. 반도체 가스센서의 모재료와 촉매를 여러가지로 바꾸거나 조합하고 센서 동작온도를 변경함으로써 선택성을 부여 하기도 한다. 많은 금속산화물 세라믹스를 가스센서의 연구 대상으로 하고 있으며, 가장 많이 연구되어 사용중인 것으로는 SnO2 , ZnO, Fe2O3 등이 있다.

아래의 테이블은 MQ 시리즈 센서의 여러 감지 기능을 요약한 것이다.

Semiconductor sensor for Flammable gas, Plastic or Metal cover		
Model	Target Gas	Datasheet
MQ-2	General combustible gas	
MQ-3	Alcohol	
MQ-4	Natural gas, Methane	
MQ-5	LPG, Natural gas, Coal gas	
MQ-6	LPG, Propane	
MQ-7	Carbon Monoxide	
MQ-8	Hydrogen	
MQ-9	CO and Combustible gas	
MQ306	LPG, Propane	
MQ307	Carbon Monoxide	
MQ303	Alcohol	
MQ131	Ozone	
MQ135	Air Quality Control	
Semiconductor sensor for Toxic gas		
MQ136	Sulfureted Hydrogen (H2S)	
MQ137	Ammonia (NH3)	
MQ138	VOC (Mellow, Benzene, Aldehyde, Ketone, Ester)	

자료출처 : 회사 홈페이지

- MQ135 – 데이터 쉬트
- https://www.olimex.com/Products/Components/Sensors/SNS-MQ135/resources/SNS-MQ135.pdf

가스센서를 사용한 경보기 제작과 사용상의 주의점

가스는 공기보다 가벼운 가스의 그룹과 무거운 가스의 그룹으로 대별된다 따라서 공기보다 가벼운 가스를 감지할 때는 천장면에 근접하게, 공기보다 무거운 가스의 경우는 바닥면에서 30cm 정도에 설치한다. 또 도시가스가 실내에서 누설된 경우 폭발 농도까지 될 때까지는 상당한 시간을 요하며 가스가 상승 또는 강하하면서 수평방향으로 확산하는 성질이 있으므로 실내에서 가스의 흐름을 차단 하는 것이 없으면 가스경보기는 가스기구나 가스설비까지의 수평거리가 4m 이내인 곳에 설치한다.

아래에 가스센서의 모양과 PinMap 을 나타내었다.

1 VCC 전원
2 AO : 아날로그 출력
3 DO 디지털 출력
4 GND

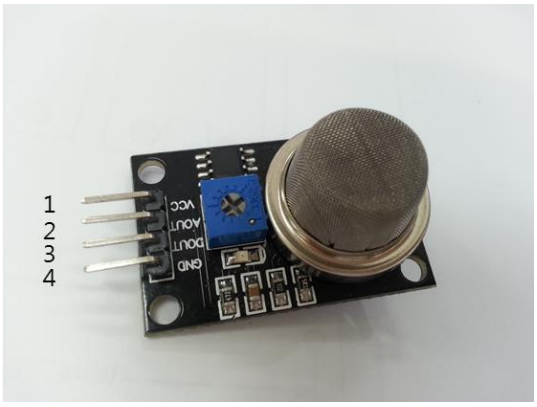
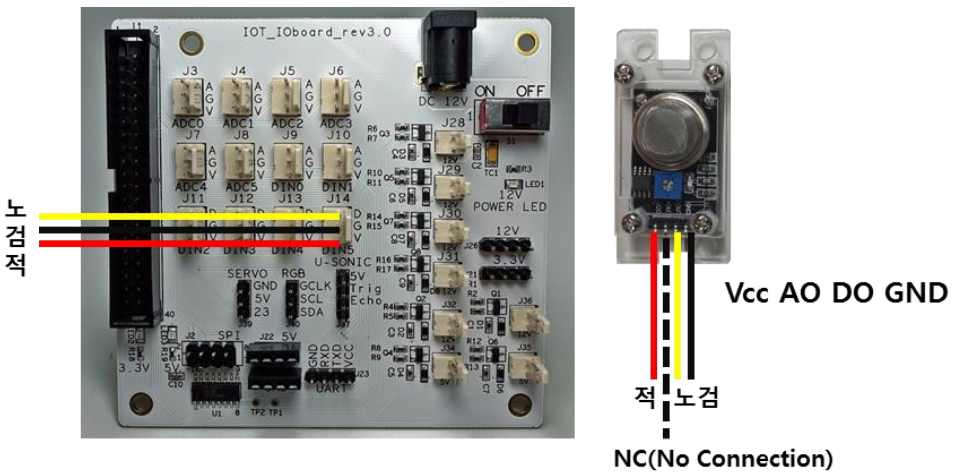


그림 21 가스 센서 외관과 핀맵



가스센서 연결도

가스센서를 테스트 하는 소스코드를 나타내었다. 가스가 감지 되지 않은경우 디지털 출력은 HIGH 에 있다가 가스가 감지되면 LOW 로 하강하는 순간에 센싱을 한다. 가스에 주의하여 시험을 진행하도록 한다.

$nano gasdetect.c 명령어로 소스를 입력한다.

```
$nano gasdetect.c


#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>


// Use GPIO Pin 17, which is Pin 0 for wiringPi library

#define MOYION_IN 0  //gpio17


// the event counter
volatile int eventCounter = 0;

// ------------------------------------------------------------------------
// myInterrupt:  called every time an event occurs
void myInterrupt(void) {
   eventCounter++;

}


// ------------------------------------------------------------------------
// main
int main(void) {
  // sets up the wiringPi library
  if (wiringPiSetup () < 0) {
      fprintf (stderr, "Unable to setup wiringPi: %s\n", strerror (errno));
      return 1;
  }

  // set Pin 17/0 generate an interrupt on high-to-low transitions
  // and attach myInterrupt() to the interrupt
  if ( wiringPiISR (MOYION_IN, INT_EDGE_FALLING, &myInterrupt) < 0 ) {
      fprintf (stderr, "Unable to setup ISR: %s\n", strerror (errno));
      return 1;
  }

  // display counter value every second.
  while ( 1 ) {
    printf( "%d\n", eventCounter );
    //eventCounter = 0;
    delay( 1000 ); // wait 1 second
  }
```

```
    return 0;
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

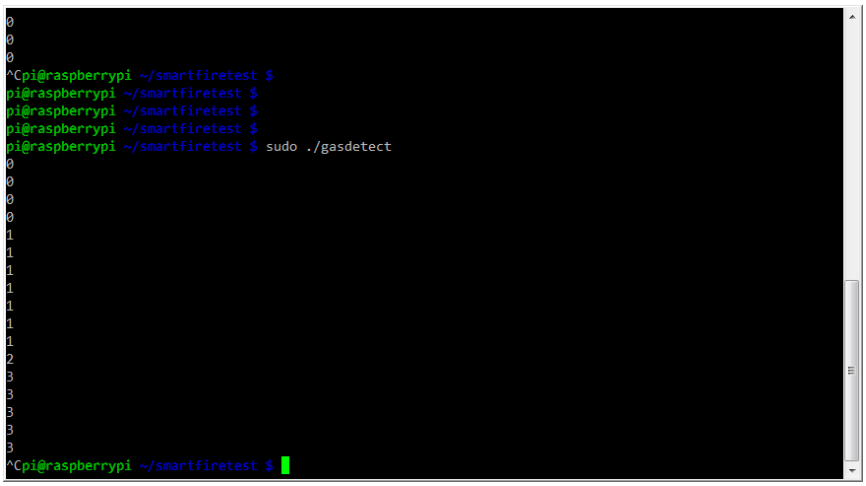
아래와 같이 컴파일 한다.

컴파일을 한다.

```
$gcc -o gasdetect gasdetect.c -lwiringPi
```

실행하여 결과를 확인한다.

```
$sudo ./gasdetect
```



가스를 흡착하여 출력을 발생시키므로 일단 가스 감지가 되면 가스가 공기중으로 날라가 안정화 될때까지 출력신호는 지연된다. 불꽃센서와 가스 센서를 시험할때는 화재, 가스에 주의해야 한다.

1.8 근접센서

인터넷 제어 토이의 근접센서는 적외선 송/수광 일체형 전방감지 센서모듈로 물체 감지시 HIGH/LOW TTL 신호를 출력한다. 거리 20~30mm 이내에 물체가 감지되면 물체를 감지하면 검출 녹색 LED 가 켜지게 된다. 한쌍의 적외선 송신 수신부가 있어서 송신 후 반사되는 적외선을 검출하여 비교기를 통하여 출력된다. 물체가 검출이 되면 출력신호는 Low level 로 떨어지며 녹색의 LED 에 불이 들어오게 되고, 검출되지 않을경우 High 출력신호를 유지한다. 형광등 불빛과 같은 외광에 강한 적외선 필터를 내장한 수광 센서가 보다 안정된 동작을 유지하는 센서이다.

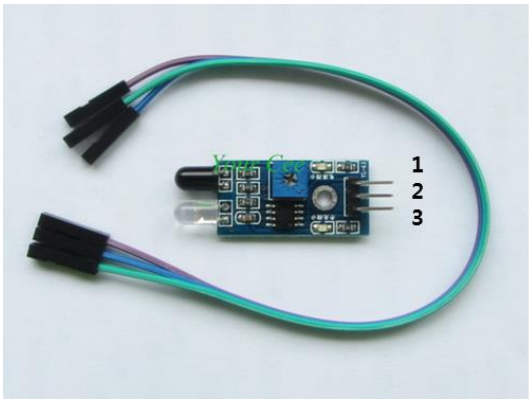
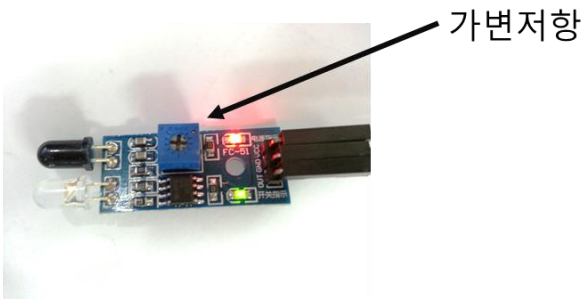


그림 22 근접센서 모양과 핀 맵

면이나 물체를 감지하고, 충돌을 사전에 감지하여 방지하는 센서로 알려져 있으며 사용하기 쉬운 장점이 있다. 초음파 센서가 2m 이내의 거리를 측정한다면 근접센서는 2~3cm 의 가까운 물체를 감지하는데 사용한다. 근접센서를 사용하기 전에 다른 센서와 마찬가지로 컬리브레이션이 필요하다.



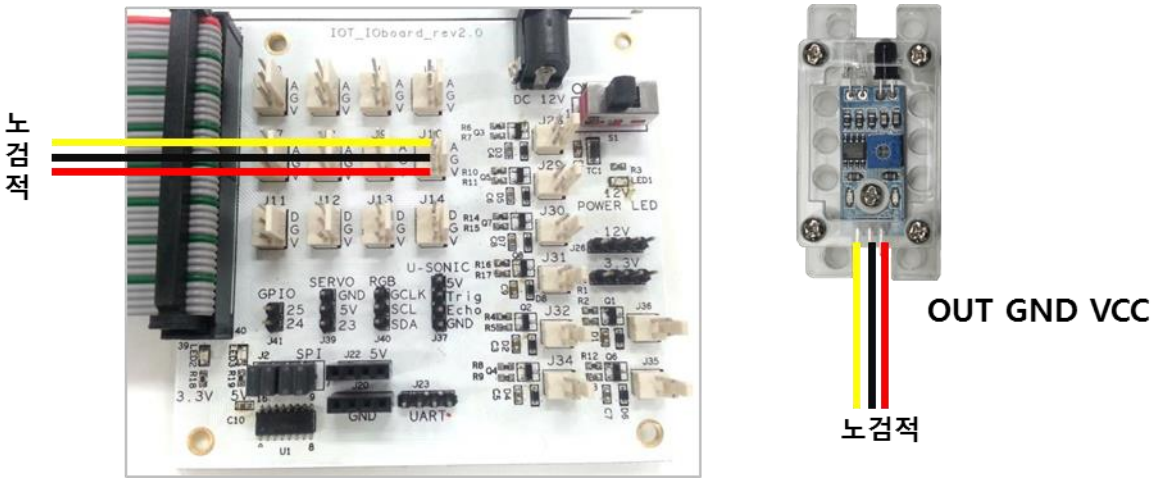
근접센서의 컬리브레이션

센서 실습을 하기 전에 [그림 136]에 나타낸 그림처럼 중앙의 가변저항을 좌 우로 회전하여 적당한 감지가 발생할 경우 LED 가 점등되는 시점에 설정하는 컬리브레이션이 반드시 필요하다. 아래 [표 19]에 근접센서의 사양을 나타내었다.

표 4 근접센서의 사양

항 목	특징
센서 원리	적외선 감지
검출 거리	20 ~ 30mm
검출 각도	중앙에서 상하 35°
비교기	LM393
입력 전원	DC 3V ~ 5V
출력 형태	디지털 출력(0, 1)

실습을 위하여 근접센서의 연결도를 [그림 138]에 첨부하였다.



근접센서의 연결도

근접센서를 실습하기 위해 nano 편집기를 이용하여 아래와 같이 소스코드를 입력한다.
$nano neardetect.c

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>


// Use GPIO Pin 27, which is Pin 0 for wiringPi library

#define COLLISION 2  //gpio27 - J13 connect


// ---------------------------------------------------------------------------
// main
int main(void)
{
   // sets up the wiringPi library
   if (wiringPiSetup () < 0)
   {
           fprintf (stderr, "Unable to setup wiringPi: %s\n", strerror (errno));
           return 1;
   }

   pinMode(COLLISION, INPUT);

   // display counter value every second.
   while ( 1 )
   {
           //printf( "%d\n", eventCounter );
           //eventCounter = 0;
```

```
            if(digitalRead(COLLISION) == 0)
                    printf("Carefull~~~~ oops \n");
            if(digitalRead(COLLISION) == 1)
                    printf("Not Collioson... \n");

            delay( 200 ); // wait 1 second
    }

    return 0;
}
```

입력을 하였다면 ctrl-x 를 누르고 저장을 위해 y 를 입력하고 같은 파일이름으로 저장하기 위해 enter 를 누르고 nano 편집기를 종료한다.

컴파일을 실행한다.

```
$ gcc -o neardetect neardetect.c -l wiringPi
```

에러 없이 컴파일이 완료되었다면 실행시켜 결과를 확인한다.

```
$ sudo ./neardetect
```

아래처럼 근점센서 앞에 아무것도 없을경우와 손이나 물체를 앞에 두었을 경우에 출력되는 메시지를 확인할 수 있다.,

```
pi@raspberrypi ~/smarttoy $ sudo ./neardetect
Not Collioson...
Not Collioson...
Not Collioson...
Carefull~~~~ oops
Carefull~~~~ oops
Not Collioson...
Not Collioson...
Not Collioson...
Carefull~~~~ oops
Carefull~~~~ oops
```

이상으로 근접센서의 실습을 마치기로 하자.

Chapter 2 액츄에이터(Actuator)

2.0 액츄에이터 실습 참고-sigint 이용

액츄에이터 제어 실습시 보통 실험을 종료할 경우 [ctrl-c] 를 눌러 종료하게 되는데, 이 때 GPIO 신호는 그대로 실습때의 상태를 유지하게 된다. 즉, 전원이 켠 상태나 전등이 ON 상태로 프로그램을 종료해도 전원이 공급되고, 전등은 ON 상태로 남아있게 되는데 이렇게 종료되는 것을 방지하는 방법을 설명한다.

컴퓨터 시스템 상에서 발생할수 있는 상황도 여러가지가 있다. 이는 OS 에 따라 다르게 설정된다. OS 에서 어떠한 시그널을 지원하는 지는 kill 명령을 이용해서 알아볼수 있다. 아래는 라즈베리 파이에서 지원하는 시그널을 캡쳐한 화면이다.

```
pi@raspberrypi ~/safeguard $ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
pi@raspberrypi ~/safeguard $ ^C
```

보통 32 번까지의 시그널은 대부분의 UNIX 계열에서 비슷한 값을 보여주며, 33 번부터 64 번까지는 아마도 OS 마다 약간식의 차이를 보여줄것이다. 33 번부터 62 번까지는 리얼타임 시스널을 위해서 예약된 시그널들이다. 각각의 시그널은 int 형의 숫자로 표시되는데, 시그널 번호에 따라서 다른 상황을 나타낸다. 각각의 시그널은 int 형의 숫자로 표시되는데, 시그널 번호에 따라서 다른 상황을 나타낸다. 모든 시그널에 대해서 설명하기엔 양이 좀 많으므로 그중 중요한 시그널만을 정리하도록 하겠다.

표 5 프로세스 시그널

시그널 이름	설명
SIGHUP	터미널을 읽어버렸을때 발생한다.
SIGABRT	프로그램의 비정상종료시 발생한다.

SIGINT	Control-C 나 DELETE 키를 입력했을때 발생한다.
SIGIO	비동기적인 입출력이 발생했을때
SIGKILL	프로세스를 죽이기 위해서
SIGPIPE	단절된 파이프에 write 할경우 발생
SIGSEGV	잘못된 메모리 참조(주로 포인터를 잘못 썼을때)
SIGSTOP	프로세스의 일시중단 (Ctrl+ z)
SIGSUSR1	사용자를 위해 정의된 시그널

.

실습시 예제를 강제로 종료하기 위해 Ctrl-c 를 누른 순간 SIGINT 신호가 발생되는데 우리는 이 신호를 이용하여 정리를 해주고 나오면 된다. 예제로 살펴본다. **(예제: sigint.c)**

```
#include <signal.h> //Signal 사용 헤더파일
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

void sig_handler(int signo); // 마지막 종료 함수

unsigned char endflag = 1;

int main()
{
    int i = 0;
    signal(SIGINT, (void *)sig_handler);

    while(endflag)
    {
        printf("%d\n", i);
        i++;
        sleep(1);
    }
    return 1;
}


void sig_handler(int signo)
{
    printf("process stop\n");
  endflag = 0;
  exit(0);
}
```

실습을 진행하면서 GPIO 핀을 초기상태(low)로 초기화, 즉 액티브 되지 않은 상태로 만들어 주고 빠져나오게 하기 위해 위 코드를 이용하면 될것이다.

2.1 워터펌프

펌프란 사전적 의미로 전기적.기계적인 에너지를 받아 유체(기체, 액체)에 힘을 가해 압력, 속도 등을 높임으로써 낮은 곳에서 높은 곳으로의 이송, 관을 통한 수송, 압력용기 등에 가압하는 기계라고 정의하고 있으나 섬유, 모래 등의 고형물을 이송하는 특수펌프도 있어 넓은 의미에서 보면 유체뿐만 아니라 고체도 취급 대상이 된다. 수처리 플랜트를 중심으로 보면 펌프는 가장 중요한 역할을 하는 기계로 물을 이송하거나 가압하는 등의 목적에 사용되며 원심펌프(Centrifugal Pump)가 약 70% 이상을 차지하고 있다. 여기서 사용하는 워터펌프는 DWP2760 이며 자세한 사양은 아래를 참고한다.



그림 23 DWP-2760 워터펌프 외관

DWP-2760 도면

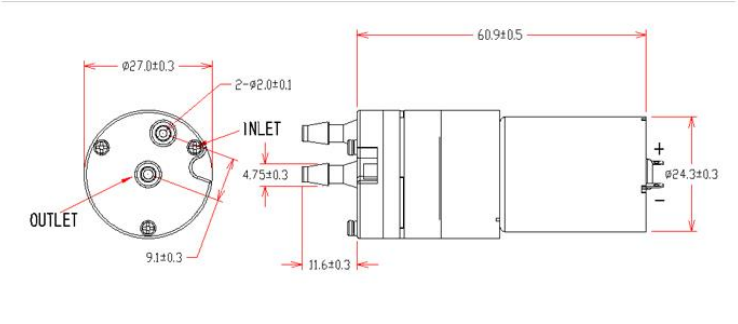


그림 24 DWP2760 워터펌프 기구도면

표 6 DWP2760 전기적 사양

정격전압	DC 12V
구동전압	DC 9V ~ DC15V
전류	무부하시 280mA 이하
최대입력	>14.5 PSI

Water Flow	≥400(±20%)mL/Min at 90℃
소음	65dB
용도	커피머신용, 90℃ 뜨거운 물 사용가능
수명	정격전류에서 50초 동작, 20초 멈춤이 1Cycle - 6,000번

아래는 워터펌프에 전원을 공급하는 회로도이며, GPIO5 번을 이용해 제어한다.

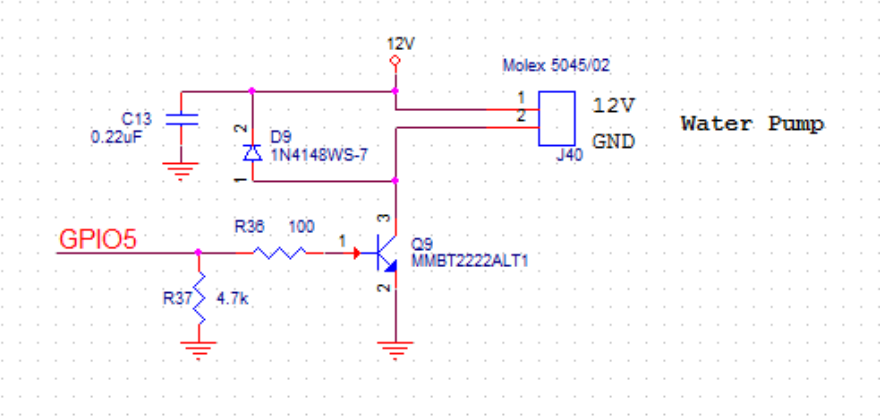


그림 25 워터펌프 구동 회로도

액츄에이터 실습시 안전하게 종료하기 위해 설명한 SIGINT 신호를 이용하여 구현하는 소스코드를 살펴보자. (pumpon.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

#define PUMP     21 // BCM_GPIO 5

void sig_handler(int signo); // 마지막 종료 함수

int main (void)
{
  signal(SIGINT, (void *)sig_handler);//시그널 핸들러 함수

  if (wiringPiSetup () == -1)
  {
        fprintf(stdout, "Unable to start wiringPi: %s\n", strerror(errno));
        return 1 ;
  }

  pinMode (PUMP, OUTPUT) ;
```

```
    for (;;)
    {
            printf("here - pump on\n");
            digitalWrite (PUMP, 1) ; // On

            delay (2000) ; // ms

            digitalWrite (PUMP, 0) ; // Off

            delay (2000) ;

    }
    return 0 ;
}

void sig_handler(int signo) //시스널 핸들러 함수 구현 //GPIO 핀 OFF 상태로 한다.
{
     printf("process stop\n");
    digitalWrite (PUMP, 0) ; // Off

    exit(0);
}
```

위와 같이 소스코드의 입력을 완료하고 ctrl-x 늘 눌러 저장하려면 y 를 누르고 파일 이름을 그대로 저장하기 위해 enter 를 눌러 빠져 나온다. 아래 명령어로 컴파일을 한다.

$gcc -o pumpon pumpon.c -lwiringPi

아래 명령어로 실행한다.

$sudo ./pumpon

워터펌프가 2 초 동작하고, 2 초 정지하는 것을 반복하는 것을 볼 수 있다. 언제나 ctrl-c 로 종료해도 워터펌프는 안전하게 종료할 것이다. 그러므로 소스코드의 pump off code 와 delay(2000) 코드들도 사실 필요가 없게 된다.

2.2 Fan

FAN 은 보통 FAN, BLOWER, 환풍기, 송풍기, 냉각팬 등 그 지칭하는 이름이 다양하며,현재 국내에서는 그 이름이 혼동되어 사용중이다. 그러나 FAN 의 용도는 분명히 다르며 용도에 따라 구분 할 수가 있다. 보통 FAN 이라고 부르는 것은 정압이 약 100mmAq 미만의 소형 FAN 을 말한다. 일반적으로 적용되는 곳은 컴퓨터, PANEL 냉각용, SMPS(파워 서플라이 냉각), 사무용 OA 기기, 인터넷장비 RACK, 용유.공압기기용 그리고 모든 PANEL 에 적용할 수 있는 전기 장치로 사용된다.



그림 26 FAN KD1205 PFS1

본 실습에서 팬은 식물농장 내부의 공기 순환용 팬으로 SLEEVE BEARING SYSTEM 구조를 가지고 있다. 12V 전원을 공급하면 회전하므로 구동방법은 단순하다. 본 실습에서 사용되는 팬의 수명은 약 30,000 시간(MTBF 상온 25℃기준)이며 제품 특징은 아래와 같다.

표 3 KD1205PFS1 특징

Size (㎜)	50*50*10mm
정격전압(VDC)	12
전류 (A)	0.1
기동전압(V)	10.8~13.8
속도 (RPM)	5500
풍량(㎥/min)	0.29
풍압(Pa)	36
소음(db)	28
권장사용온도(℃)	10~60
무게(g)	20

팬 사용시는 전류와 회전수와 같은 물리적 특성에 주의해서 사용해야 하고, 바람의 흡 배기 방향에 맞도록 설치하여 사용해야 한다. 아래는 팬의 기구도면을 나타내었다. 바람의 방향과 팬의 사이즈를 나타내었다.

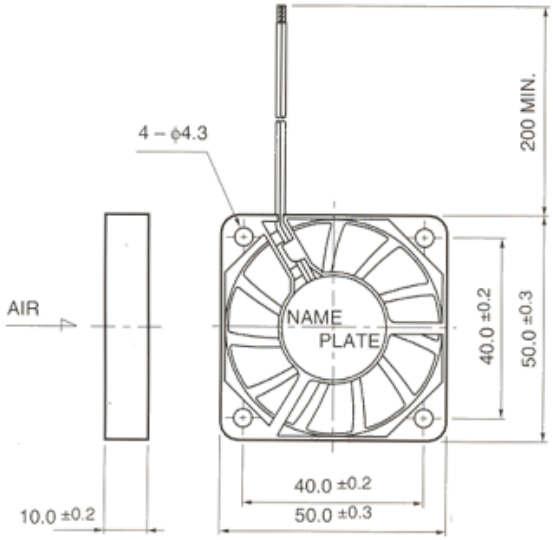


그림 27 팬의 기구도면

아래는 팬의 전원을 제어하는 회로도이며, GPIO 6 번을 사용하여 전원을 제어한다.

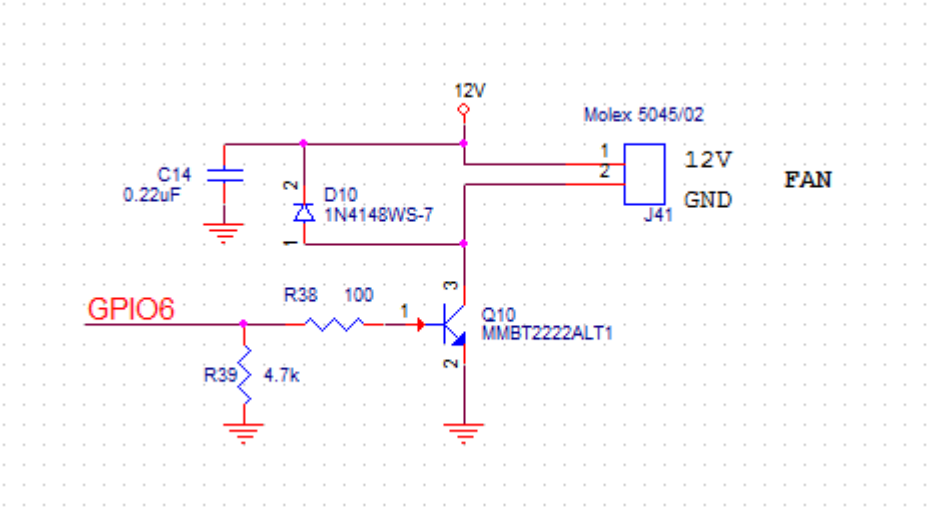


그림 28 팬 구동 회로도

아래는 디레이를 삭제하고 ctrl-c 로 종료할때를 보완한 소스코드이다. 워터펌프를 실습할때와 동일한 코드이다. (fanon.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

#define FAN      22 // BCM_GPIO 6
```

```
void sig_handler(int signo); // 마지막 종료 함수

int main (void)
{
  signal(SIGINT, (void *)sig_handler);//시그널 핸들러 함수

  if (wiringPiSetup () == -1)
  {
        fprintf(stdout, "Unable to start wiringPi: %s\n", strerror(errno));
        return 1 ;
  }

  pinMode (FAN, OUTPUT) ;

  for (;;)
  {
        //printf("here - FAN on\n");
        digitalWrite (FAN, 1) ; // On
  }
  return 0 ;
}

void sig_handler(int signo)
{
    printf("process stop\n");
  digitalWrite (FAN, 0) ; // Off

  exit(0);
}
```

소스코드의 입력을 완료하고 ctrl-x 늘 눌러 저장하려면 y 를 누르고 파일 이름을 그대로 저장하기 위해 enter 를 눌러 빠져 나온다. 아래 명령어로 컴파일을 한다.

```
$gcc -o fanon fanon.c -lwiringPi
```

아래 명령어로 실행하여 결과를 확인한다.

```
$sudo ./fanon
```

2.3 DC Motor

DC 모터란, 고정자로 영구자석을 사용하고, 회전자(전기자)로 코일을 사용하여 구성한 것으로, 전기자에 흐르는 전류의 방향을 전환함으로써 자력의 반발, 흡인력으로 회전력을 생성시키는 모터이다. 모형 자동차, 무선조정용 장난감 등을 비롯하여 여러 방면에서 가장 널리 사용되고 있는 모터이다.

DC 모터는 다음과 같은 특징이 있다. (1) 기동 토크가 크다. (2) 인가전압에 대하여 회전특성이 직선적으로 비례한다. (3) 입력전류에 대하여 출력 토크가 직선적으로 비례하며, 또한 출력 효율이 양호하다. (4) 가격이 저렴하다.

DC 모터의 가장 큰 결점으로는 그 구조상 브러시(brush)와 정류자(commutator)에 의한 기계식 접점이 있다는 점이다. 이것에 의한 영향은 전류(轉流)시의 전기불꽃(spark), 회전 소음, 수명이라는 형태로 나타난다. 그리고, 마이크로컴퓨터 제어를 하려는 경우는 "노이즈"가 발생하게 된다. 따라서 이 노이즈 대책이 유일한 과제가 될 수 있다. 이 노이즈 대책을 위해서는 각 단자와 케이스 사이에 0.01μF~0.1μF 정도의 세라믹 콘덴서를 직접 부착한다. 이것으로 정류자에서 발생하는 전기불꽃을 흡수하여 노이즈를 억제할 수 있다. 본 실습에 사용되는 DC 모터는 비닐 하우스나 가림막등을 열고, 닫는 용도로 사용한다고 가정한다.

표 7 DC Motor Specification

품명	마이크로 DC 모터
Rated Voltage(V)	12
No load speed(rpm)	9,100
No load current(mA)	60
Rated speed(rpm)	7800
Rated torque(gf.츠)	20
Rated current(mA)	350
Size	23.8Φ x 30.5mm
Noise level(유)	55

DC 모터를 선택할때에는 토크(회전력)와 회전수를 고려하여 사용에 필요한 용량의 1.5 배 정도 되는 모터를 사용해야 한다. 아래는 본 실습에 사용되는 DC 모터의 외관을 나타내었다.



그림 29 DC 모터 외관

아래는 본 실습에 사용되는 DC 모터의 기구 도면을 나타내었다. 기구물에 장착하거나 나사로 고정을 할 때 참고하기 위함이다.

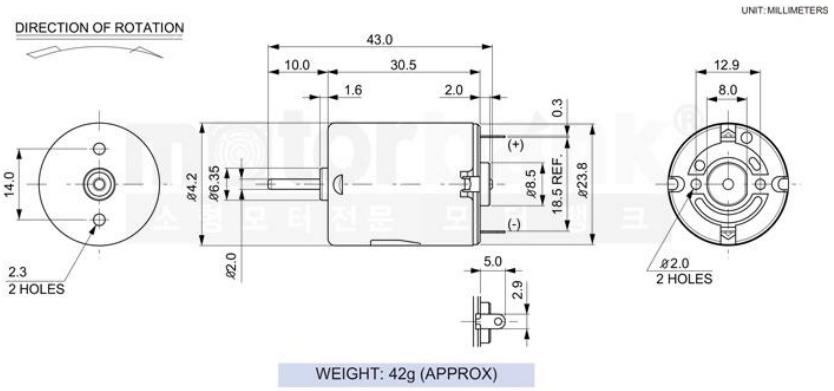


그림 30 DC motor 기구 도면

아래 [그림 35]에 DC 모터에 전원을 공급하는 회로도를 나타냈다. TR 을 이용하여 DC 모터에 공급되는 전원을 On/Off 한다. 보통 TR 을 사용할 경우 TR 의 전력 손실은 그대로 열로 발생되기 때문에 방열판이나 냉각팬등을 사용하는 방식으로 열 대책을 하여야 한다. 모터의 On/Off 제어는 여기서는 단방향이지만 회전의 방향을 바꾸고 싶을 때는 모터에 인가하는 전압의 플러스, 마이너스를 반대로 하면 모터는 역회전하게 된다. 그래서, 단일전원으로 모터에 가하는 전압의 방향을 바꿀 수 있는 회로로 고안된 것이 "H 브리지 회로"라고 한다.

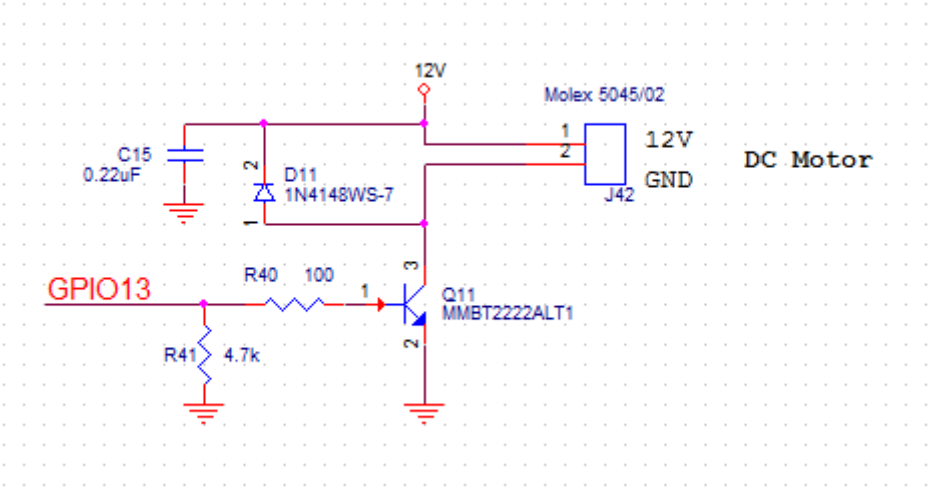


그림 31 DC motor 구동 회로도

아래 소스코드는 DC 모터를 제어하는 소스코드이다. 전원 On-Off 로 제어만 하면 회전 속도도 빠르고 소음이 많이 발생하므로 wiringPi 의 라이브러리를 이용하여 PWM 방식으로 제어하는 코드이다. (motorsoftpwm.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일
```

```
#include <wiringPi.h>
#include <softPwm.h>

#define MOTORCONTROL  23//GPIO 13 - Motor

void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
  if(wiringPicheck()) printf("Fail\n");

  signal(SIGINT, (void *)sig_handler);

  pinMode (MOTORCONTROL, OUTPUT);

  softPwmCreate(MOTORCONTROL, 0, 100);

  int i;
  int delaytime = 100;
  while(1)
  {
         printf("here \n");

         softPwmWrite(MOTORCONTROL, 5);
         delay(delaytime);

  }
 return 0 ;
}

int wiringPicheck(void)
{
  if (wiringPiSetup () == -1)
  {
         fprintf(stdout, "Unable to start wiringPi: %s\n", strerror(errno));
         exit(1) ;
  }
}

void Bpluspinmodeset(void)
{
  pinMode (MOTORCONTROL, OUTPUT);
}

void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
    printf("process stop\n");
  digitalWrite(MOTORCONTROL, 0);
  exit(0);
}
```

소스코드의 입력을 완료하고 ctrl-x 늘 눌러 저장하려면 y 를 누르고 파일 이름을 그대로 저장하기 위해 enter 를 눌러 빠져 나온다. 아래의 명령을 사용하여 컴파일 한다.

```
$ gcc -o motorsoftpwm motorsoftpwm.c -lwiringPi
```

아래의 명령을 사용해 실행하면 낮은 속도로 회전하는 모터를 보게 된다.

```
$ sudo ./motorsoftpwm
```

2.4 RGB Led

RGB led 모듈은 IWS-506-RGB-K3 led 3 개를 연결하여 제작되었다. 전원은 DC 12V 를 공급하고, GPIO 3 라인을 연결하여 다양한 색을 출력하도록 한다. RGB Led 가 내는 색상은 Red, Green, Blue 각각 256 가지 색을 내므로 조합하면 1,670 만 가지 색을 낼 수 있다. 여기서는 RGB 색을 시험하기로 한다. 아래는 RGB Led 의 이미지와 회로도를 나타낸다.

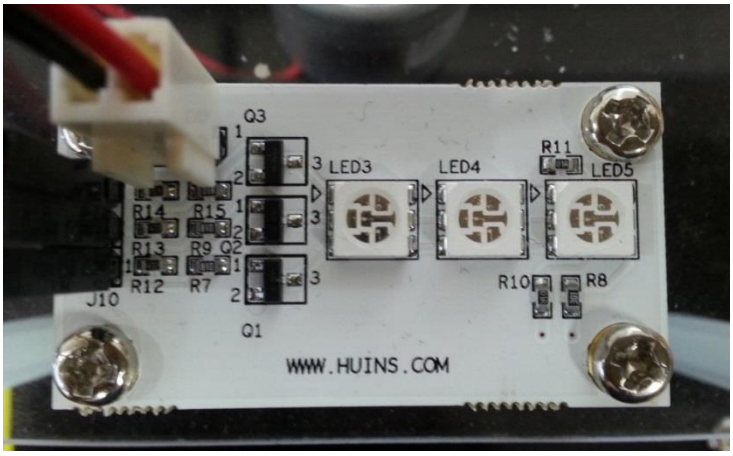


그림 32 RGB Led 모듈 외형

아래 [그림 36]은 GRB Led Moule 회로도를 나타낸다.

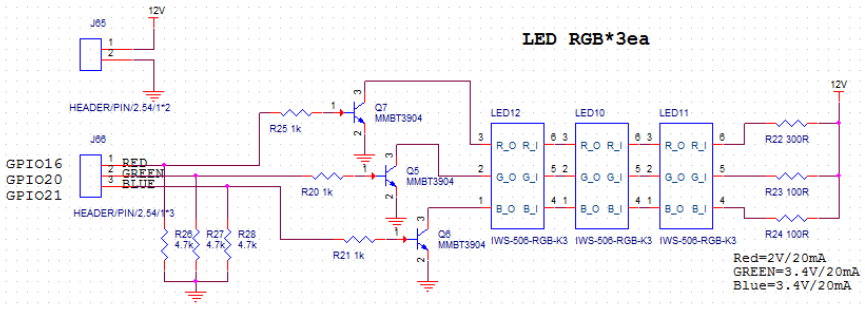


그림 33 RGB Led 모듈 회로도

아래는 [그림 45]은 GRB Led Moule 에 들어가는 전원을 제어하는 회로도이다.

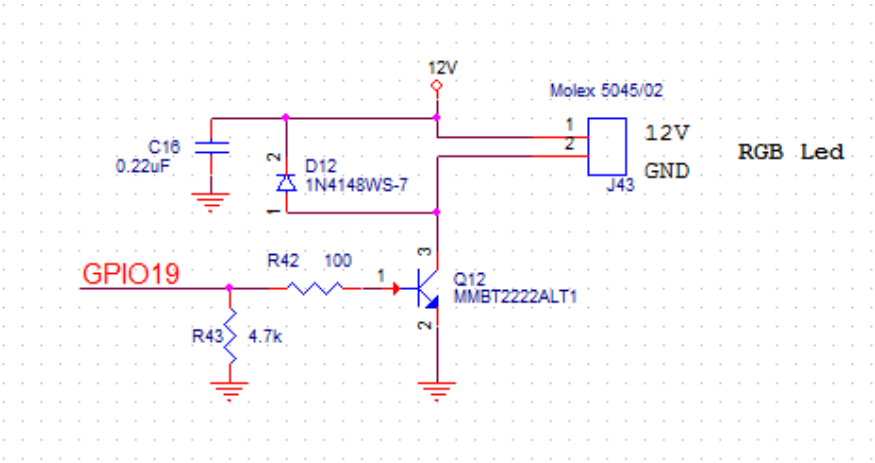


그림 34 RGB Led 모듈 전원 회로

RGB LED 를 테스트 하는 소스코드를 나타낸다. 마찬가지로 ctrl-c 를 눌러 종료할 때 정리하는 코드가 들어가 있다. GPIO 핀 번호를 정의하고, 핀 의 방향을 정해주고 1 초 마다 red, blue, green 색을 교대로 점등하는 간단한 프로그램이다. (rgbtest.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

#define RGBLEDPOWER  24 //BCM_GPIO 19

#define RED             27
#define GREEN    28
#define BLUE     29

void sig_handler(int signo); // SIGINT 사용 마지막 종료 함수

int main (void)
{
  signal(SIGINT, (void *)sig_handler); //시그널 핸들러 함수

  if (wiringPiSetup () == -1)
  {
        fprintf(stdout, "Unable to start wiringPi: %s\n", strerror(errno));
        return 1 ;
  }

  pinMode(RGBLEDPOWER, OUTPUT);
  pinMode(RED, OUTPUT);
  pinMode(GREEN, OUTPUT);
  pinMode(BLUE, OUTPUT);

  int i;
```

```
  for (i =0; i<10 ;i++)
  {

        digitalWrite(RGBLEDPOWER, 1);

        digitalWrite(RED, 1);
        digitalWrite(BLUE, 0);
        digitalWrite(GREEN, 0);

        delay(1000);

        digitalWrite(RED, 0);
        digitalWrite(BLUE, 1);
        digitalWrite(GREEN, 0);

        delay(1000);

        digitalWrite(RED, 0);
        digitalWrite(BLUE, 0);
        digitalWrite(GREEN, 1);

        delay(1000);
  }
  return 0 ;
}

void sig_handler(int signo)
{
    printf("process stop\n");

  digitalWrite(RED, 0);
  digitalWrite(GREEN, 0);
  digitalWrite(BLUE, 0);
  digitalWrite(RGBLEDPOWER, 0); //Off

  exit(0);
}
```

소스코드의 입력을 완료하고 ctrl-x 늘 눌러 저장하려면 y 를 누르고 파일 이름을 그대로 저장하기 위해 enter 를 눌러 빠져 나온다. 아래의 명령을 사용하여 컴파일 한다.

```
$ gcc -o rgbtest rgbtest.c -lwiringPi
```

아래 명령어로 실행한다.

```
$ sudo ./rgbtest
```

RGB 컬러 전체 표시하기 실습

다음은 소프트웨어 PWMdm,f 이용하여 RGB 전체 색을 표시하는 실습을 할것이다. 실제로 1,600 만 가지 이상의 색을 표현하지만 우리가 구별하지 못할 뿐이다.

LED 로 여러가지 색을 나타내려면 기본적으로 LED 의 밝기를 조절해야하는데 LED 밝기 조절은 전압을 변경하여 할 수 있다. 하지만 GPIO 핀은 디지털인데, 즉, 끄고 켜는 두가지 외에는

방법이 없는데 어떻게 LED 의 밝기를 조절 할 수 있을까? 이를 위해서는 PWM 을 이해해야한다. PWM 은 Pulse Width Modulation 의 약자로 펄스 폭 변조라고 한다. PWM 은 디지털 기기에서 아날로그 결과를 얻기 위한 테크닉으로, 전원인 on 되는 시간을 펄스 폭이라고 하고, 펄스 폭을 조정함으로 on 과 off 사이의 전압을 흉내낼 수 있다.

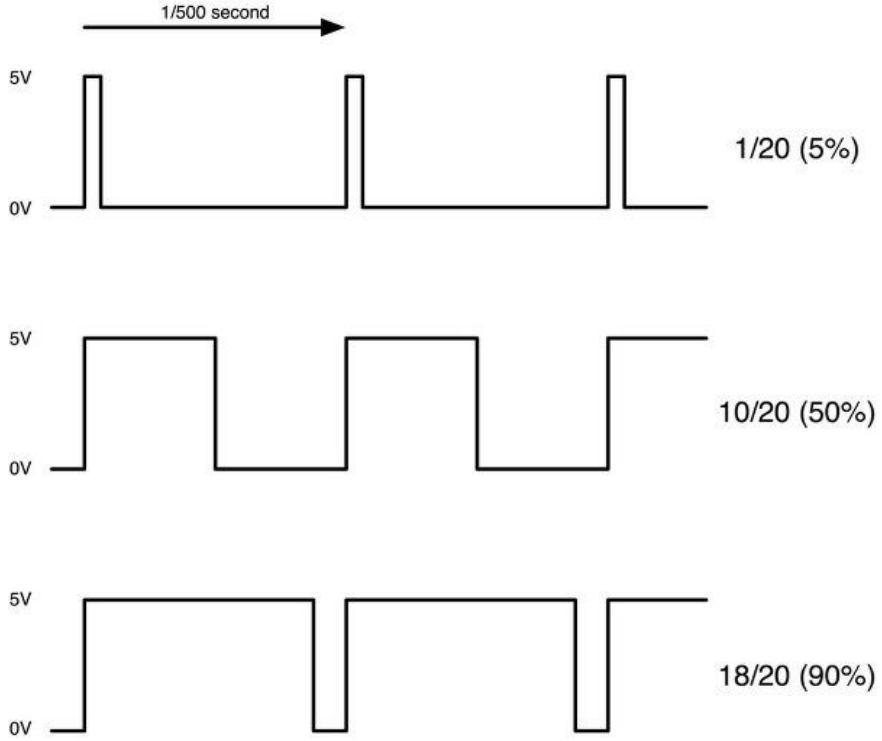


그림 35 PWM 신호 원리

이러한 on-off 패턴을 빠르게 반복하여 LED 에 출력하면 LED 의 밝기를 변경 할 수 있게 되는 것이다. 예를 들어 위의 [그림 39]와 같이 on 이 지속 되는 시간, 즉, 펄스 폭을 조정하면, 위에서 아래로 갈 수록 밝은 LED 빛을 얻을 수 있다.

wiringPi 라이브러리에서 PWM 을 지원한다. 다음은 WiringPi 라이브러리의 PWM 함수를 이용해서 LED 의 색깔을 바꿔서 다양한 색을 표현하는 프로그램 코드이다. (allrgbcolor.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>
#include <softPwm.h>

#include <wiringPi.h>

#define RGBLEDPOWER  24 //BCM_GPIO 19

ㅣㄴ
#define RED             29     // BCM_GPIO 16 - OUT
#define GREEN    28     // BCM_GPIO 20 - OUT
```

```
#define BLUE     27             // BCM_GPIO 21 - OUT

void Bpluspinmodeset(void);

void setRGB(int r, int g, int b);
void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
  if(wiringPicheck()) printf("Fail");

  Bpluspinmodeset();

  signal(SIGINT, (void *)sig_handler);

  int i;
  digitalWrite(RGBLEDPOWER, 1);

  while(1)
  {

         int i = 0, j = 0, k=0;
         printf("RGB LED Various Color");

         softPwmCreate(RED, 0, 255);
         softPwmCreate(GREEN, 0, 255);
         softPwmCreate(BLUE, 0, 255);

         for (i = 0; i <= 250; i=i+10){
                for (j = 0; j <= 250; j=j+10){
                       for (k = 0; k <= 250; k=k+10){
                              setRGB(i,j,k);
                              delay(50);
                              printf("R:%d G:%d B:%d\n", i, j, k);
                       }
                }
         }

         setRGB(0,0,0);//clear LED
         delay(50);
  }

  return 0 ;
}

int wiringPicheck(void)
{
  if (wiringPiSetup () == -1)
  {
         fprintf(stdout, "Unable to start wiringPi: %s\n", strerror(errno));
         return 1 ;
  }
}

void Bpluspinmodeset(void)
{
```

```
   pinMode(RGBLEDPOWER, OUTPUT);
   pinMode (RED, OUTPUT);
   pinMode (GREEN, OUTPUT);
   pinMode (BLUE, OUTPUT);

}

void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
    printf("process stop\n");
   digitalWrite(RED, 0);
   digitalWrite(GREEN, 0);
   digitalWrite(BLUE, 0);
   digitalWrite(RGBLEDPOWER, 0); //Off
   exit(0);
}

void setRGB(int r, int g, int b){
  softPwmWrite(RED, 255-r);
  softPwmWrite(GREEN, 255-g);
  softPwmWrite(BLUE, 255-b);
}
```

소스입력을 다 했으면 ctrl-x 를 누르고 y 를 누르고 enter 를 입력하여 nano 에디터를 빠져나온다. 위 코드를 조금 살펴보자. 우선 8 번째 줄을 보면 'softPwm.h' 헤더파일을 포함시키는데, 이는 WiringPi 에서 지원하는 헤더파일로 PWM 을 사용하기 위해서는 이 헤더파일이 필요하기 때문이다. 두번째로 36-38 번째 줄을 보면, softPwmCreate 함수를 사용하는데 이 함수는 소프트웨어로 제어되는 PWM 핀을 만든다. 이 함수는 핀번호, 기본값, PWM 값의 범위 이렇게 세가지를 파라미터로 갖는다. 따라서 36 번째 줄의 softPwmCreate(RED, 0, 255) 는 RED 핀을 0 과 255 사이의 범위를 갖고 기본값을 0 으로 하는 PWM 핀을 만든다. 마지막으로 void setRGB(int r, int g, int b) 함수는 LED 에 불을 켜는 함수로 softPwmWrite 를 이용해서 빨간색, 초록색, 파란색 불빛의 밝기를 조절하여 원하는 색깔의 빛을 만든다.

아래 명령어로 컴파일을 한다.

```
$ gcc -o allrgbcolor allrgbcolor.c -lwiringPi
```

아래 명령어로 실행을 시키면 천천이 색이 변하는 것을 볼 수 있다. For 루프의 증가분을 바꿔주면 더욱 빠르게 변하게 된다.

```
pi@raspberrypi ~/smartfarm $ sudo ./allrgbcolor
```

2.5 LED Bar 실습

LED Bar 는 단순히 화이트 LED3 개를 연결하여 구성한 것이다. 흔히 조명으로 사용되는 LED 들이 대부분 Bar 로 구성되어 있다. Led 소자의 자세한 사양은 데이터북을 참조한다.
(http://www.snowdragonledhk.com/download/5050-smd-led/top-led-5050-smd-white-led-superbright.pdf)

표 8 Led 사양

ENG. N.O	120203002
Shape	8MM Round 5.0*5.4*1.6mm Surface Mount LEDs
Emitting Color	White
Lens Color	Yellow
Viewing Angle	117°
Chip Material	InGaN

LED bar 는 실내조명, 실외조명, 특수조명 등과 같은 많은 응용분야가 있다. 여기서는 실내조명을 ON/OFF 하는 기능을 한다.



그림 36 LED bar 외관

LED bar 의 연결 회로도를 나타낸다.

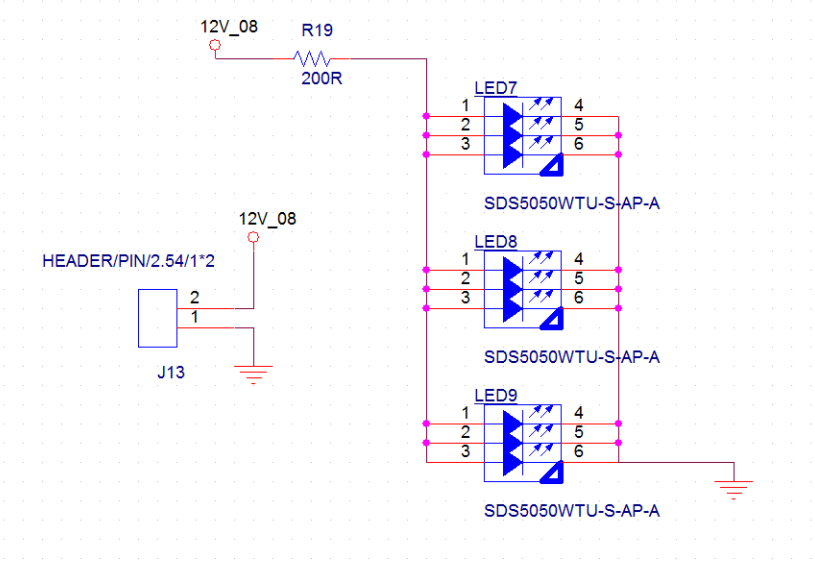


그림 37 LED bar 회로도와 전원공급 커넥터

조명 LED 를 시험 하기 위해 소스 코드를 입력한다. (lightcontrol.c) 이 코드에서는 앞에서 소개한 Signal 을 사용하는 방법을 포함하여 코딩하였다. 실습중 전등 LED 가 켜진채 ctrl-c 로

종료하여도 전등이 켜져있는 일은 없을 것이다. 물론 다른 액츄에이터 제어 프로그램에서도 적용시켜 프로그램 하면 실수로 액츄에이터를 장시간 작동하게 하는 실수를 줄일 수 있다.

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

#define LIGHTCONTROL  21//GPIO 5

void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
  if(wiringPicheck()) printf("Fail\n");

  signal(SIGINT, (void *)sig_handler);

  int i;
  while(1)
  {
         printf("here \n");

         pinMode (LIGHTCONTROL, OUTPUT);

         //delay(1000);

         digitalWrite(LIGHTCONTROL, 1);
         delay(1000);
         digitalWrite(LIGHTCONTROL, 0);
         delay(1000);


  }

 return 0 ;
}

int wiringPicheck(void)
{
  if (wiringPiSetup () == -1)
  {
         fprintf(stdout, "Unable to start wiringPi: %s\n", strerror(errno));
         return 1 ;
  }
}

void Bpluspinmodeset(void)
{
  pinMode (LIGHTCONTROL, OUTPUT);

}
```

```
void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
    printf("process stop\n");
  digitalWrite(LIGHTCONTROL, 0);
  exit(0);
}
```

소스를 입력하였으면 아래 명령어로 컴파일을 수행한다

$gcc -o lightcontrol lightcontrol.c -lwiringPi

컴파일 하고 실행하면 조명이 온 오프 되는 것을 확인 할 수 있다.

$sudo ./lightcontrol

실행중 ctrl-c 로 프로그램이 종료되어도 전등이 켜져 있는 일은 없을 것이다.

PWM 으로 LED bar 제어하기

간단한 LED 제어실습으로 softPwm 라이브러리를 사용하여 LED bar 를 제어하는 방법을 알아보자. PWM 은 디지털 기기에서 아날로그 결과를 얻기 위한 테크닉으로, 전원인 on 되는 시간을 펄스 폭이라고 하고, 펄스 폭을 조정함으로 on 과 off 사이의 전압을 흉내낼 수 있다. 이러한 on-off 패턴을 빠르게 반복하여 LED 에 출력하면 LED 의 밝기를 변경 할 수 있게 되는 것이다. softPwm 라이브러리나 헤더파일은 라즈베리 파이 실습시에 설치한 wiringPi 라이브러리가 설치되었다면 pi₩wiringPi₩wiringPi 아래에 있을것이다.

우선 소스코드를 살펴보면 아래와 같다. (ledbarsoftpwm.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>
#include <softPwm.h>

//#define MOTORCONTROL  22     //GPIO 6 - Motor

#define LEDCONTROL  21// LED Bar
//const int LEDCONTROL 21 //

void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
  if(wiringPicheck()) printf("Fail\n");

  signal(SIGINT, (void *)sig_handler);

  pinMode (LEDCONTROL, OUTPUT);

  softPwmCreate(LEDCONTROL, 0, 100);

  int i;
  int delaytime = 100;
  while(1)
```

```
  {
        printf("here \n");

        softPwmWrite(LEDCONTROL, 10);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 20);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 30);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 40);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 50);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 60);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 70);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 80);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 90);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 100);
        delay(delaytime);
        softPwmWrite(LEDCONTROL, 0);
        delay(delaytime);
  }

  return 0 ;
}

int wiringPicheck(void)
{
  if (wiringPiSetup () == -1)
  {
        fprintf(stdout, "Unable to start wiringPi: %s\n", strerror(errno));
        exit(1) ;
  }
}

void Bpluspinmodeset(void)
{
  pinMode (LEDCONTROL, OUTPUT);

}

void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
    printf("process stop\n");
  digitalWrite(LEDCONTROL, 0);
  exit(0);
}
```

wiringPi 의 소프트웨어 pwm 으로 사용하기 위해 아래의 두 파일을 include 한다.

```
#include <wiringPi.h>
```

```
#include <softPwm.h>
```

softPwm 은 최소 10usec 의 펄스 길이와 100hz 의 PWM frequency 를 제공하므로 모터와 LED 제어를 이해 사용하는데 적당하다.

softPwmCreate 함수를 사용하는데 이 함수는 소프트웨어로 제어되는 PWM 핀을 만든다. 이 함수는 핀번호, 기본값, PWM 값의 범위 이렇게 세가지를 파라미터로 갖는다.

softPwm(use pin, init value, range pwm);

실제로 정의된 핀에 PWM 신호를 출력하기 위해 핀 번호와 값을 인자로 갖는 아래 함수를 사용한다.

softPwmWrite(use pin, pwm value);

현재 pwm 출력값의 변화로 LED Bar 는 밝기가 밝아지고 꺼지고 하는 동작을 반복할 것이다.

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

소스코드를 이해하고 입력을 하였다면 컴파일을 수행한다.

```
$ gcc -o ledbarsoftpwm ledbarsoftpwm.c -lwiringPi -lpthread
```

실행을 하면 LED 조명의 밝기가 변화하는 것을 볼 수 있다.

```
$ sudo ./ledbarsoftpwm
```

이러한 PWM 제어는 모터, LED 조명의 밝기제어, RGB Color LED 의 색 제어 등에 많이 사용되므로 잘 알아두면 매우 유용할 것이다

2.6 Buzzer(부저) 실습

무인 경비 시스템에 사용되는 부저는 침입자를 감지할 경우 싸이렌과 같은 큰 소리를 내는 부저이다. 실제 소리는 귀가 따가울 정도의 소리를 내어 학습에 지장을 초래하므로 여기서는 5V 에 동작하는 피에조 부저를 이용 하기로 한다.



그림 38 부저(멜로디) 외관

부저는 내부의 PIEZO 를 이용하여 소리를 내는 제품이며, 영구자석보다 더 강한 자력에 노출되는 경우, 자력이 손실되어 제품 성능에 이상이 생길수 있다. 방수 제품이 아니므로, 플럭스나 솔벤트 등의 과다 사용으로 이물질이 내부에 침투시에는 소리에 이상이 있을수 있으니 주의하여야 한다.

아래와 같이 소스코드를 입력한다. (buzcontrol.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

//#define BUZCONTROL  27//GPIO 5
#define BUZCONTROL  26  //GPIO 12

void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
  if(wiringPicheck()) printf("Fail\n");

  signal(SIGINT, (void *)sig_handler);

  int i;
  while(1)
  {
        printf("here \n");

        pinMode (BUZCONTROL, OUTPUT);

        //delay(1000);

        digitalWrite(BUZCONTROL, 1);
        delay(1000);
        digitalWrite(BUZCONTROL, 0);
        delay(1000);


  }

 return 0 ;
}

int wiringPicheck(void)
{
  if (wiringPiSetup () == -1)
  {
        fprintf(stdout, "Unable to start wiringPi: %s\n", strerror(errno));
        return 1 ;
  }
```

```
}

void Bpluspinmodeset(void)
{
  pinMode (BUZCONTROL, OUTPUT);

}

void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
    printf("process stop\n");
  digitalWrite(BUZCONTROL, 0);
  exit(0);
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

```
$gcc -o buzcontrol buzcontrol.c -lwiringPi
```

실행을 시키면 삐 소리가 들리면 실습이 잘 된것이다.

```
$sudo ./buzcontrol
```

2.7 서보모터 실습

서보모터는 별도의 부가회로 없이 간단한 로봇 콘트롤러에 접속해서 사용할수 있으며, 로봇 컨트롤러를 사용하면 -90°~+90° 의 범위내에서 원하는 각도로 제어가 가능하기 때문에 로봇의 관절 동작을 쉽게 제어 할수 있다.

표 9 서보모터 HES-288 사양 및 특징

서보종류	아날로그 서보모터
동작전압	4.8V ~ 6V
토크(Kg.cm)	2.5Kg / cm
스피드	0.2ch/60도
크기	
무게(g)	50g 이하
기어재질	프라스틱
특징	표준서보
회전각	180도(-90도 ~ +90도)



그림 39 서보모터의 외형

대부분의 RC 서보 모터의 선은 3 선으로 제어하는데 [검은선(갈색) : 그라운드], [적색선(빨강) : 전원(4.8~6V)], [노란선(주황) : 제어신호선(각도제어)] 이렇게 사용된다. 서보 모터의 3 개의 선은 신호 기능에 맞게 사용하여야 하며, 잘못 연결되면 서보 모터 내부의 회로가 손상을 입을수 있게 된다. 또 모터의 규정각도 이상으로 무리하게 움직이게 되면 내부의 기어가 망가지게 되므로 주의한다.

서보모터의 사용방법은 아래 그림과 같이 1.5msec 일 때는 중심각도(0)로 0.7msec 일 때는 오른쪽(+ 90), 2.3msec 일 때는왼쪽(-90)으로 동작한다. 서보모터로 현재의 위치를 계속 해서 유지하기 위해서는 위치 신호(PWM)를 10msec~ 20msec 정도 반복해서 보내 주어야 한다. 신호를 받지 못한 상태에서는 이 시간이 지나면, 서보 모터가 정지하게 되며, 따라서 정해진 위치를 벗어나게 되어도 이를 자동으로 복구하지 못하게 된다.

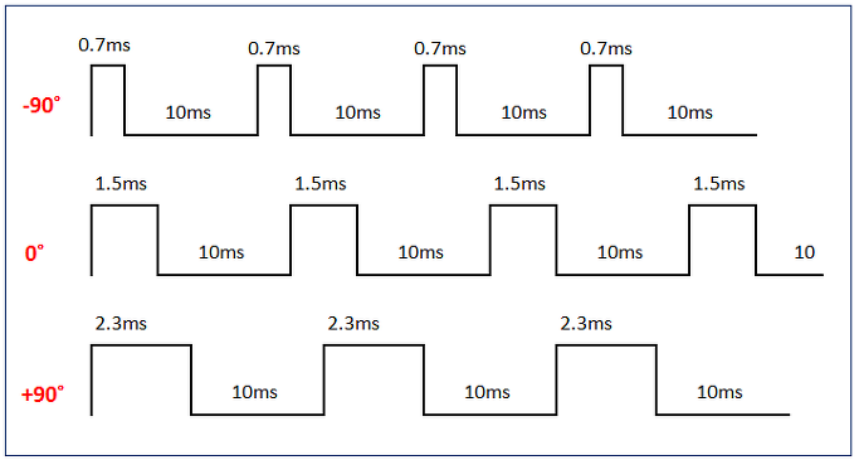


그림 40 서보모터의 PWM 신호

서보모터의 연결은 아래 그림과 같이 빨간색 선에 4~6V 전원을 공급 하고 주황색 선에는 제어신호(PWM)를 인가하고 갈색에는 GND 를 연결한다. 한가지 주의할 점은 노란색 선에

공급되는 제어신호 (PWM)의 신호의 펄스는 실제의 7~23%의 오차 범위에 있어야 정확한 제어를 할 수 있다.

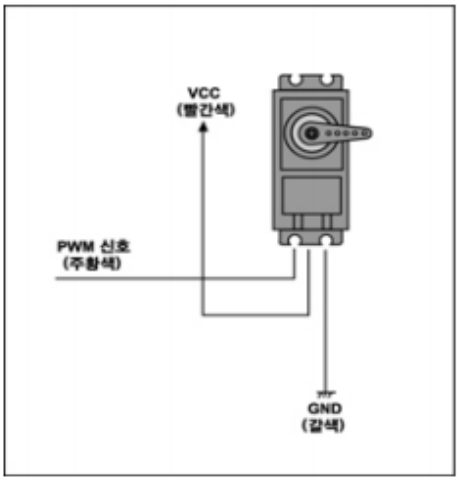


그림 41 서보모터 연결방법

RC 서보모터는 별도의 드라이버 칩을 사용하지 않는다. 다만, PWM 을 이용하기 위해서는 라즈베리 파이의 PWM 핀인 GPIO18 번을 이용하든지, 아니면 별도의 서프트웨어 방식의 PWM 을 사용하여야 한다. 여기서는 wiringPi 에서 제공하는 소프트웨어 PWM 방식을 사용하기로 한다.

작업 폴더를 생성하자.

```
$mkdir rcservotest
$cd rcservotest
```

우선 nano 편집기를 사용하여 소스코드를 작성한다. ($nano rcservo.c)

```
$nano rcservo.c

#include <stdio.h>
#include <stdlib.h>
#include <wiringPi.h>

#include "softServo.h"

#define RCSERVO  5      //GPIO 5

int main(void)
{
  if (wiringPiSetupGpio() == -1)
    return 1 ;

  softServoSetup(RCSERVO,-1,-1,-1,-1,-1,-1,-1);

  printf("rc servo test \n");

  int i=0;
```

```
  for( i= 0; i<10 ; i++){
        printf("rc servo test \n");
        softServoWrite(RCSERVO, -250);
        delay(1000);

        softServoWrite(RCSERVO, 1250);
        delay(1000);

  };
}
```

위에서 호출하는 softServoWrite 함수가 softServo.c 에 있는 파일이다. 이 파일은 wiringPi 라이브러리 아래에 있다. softServo.c 와 softServo.h 를 복사하자.

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

현재 디렉토리를 나타내는 명령어는 $pwd 이다.

```
pi@raspberrypi ~/rcservotest $ pwd
/home/pi/rcservotest
```

프로그램을 빌드하면

```
$gcc -o rcservotest rcservotest.c softServo.c -lwiringPi
```

프로그램을 실행하자.

```
$sudo ./rcservotest
```

프로그램이 실행되면 정방향으로 1 초동안 회전하고 다시 역방향으로 1 초간 회전한다. 사용자의 설계대로 회전하고 규칙적으로 움직이도록 프로그램을 구성하면 된다.

chapter 3 참고자료

Smart IoT 설계 시스템 사용자 설명서는 많은 자료와 인터넷 사이트를 참고하여 작성되었다. 그만큼 자료의 출처와 자료검토를 신중하게 숭행하였다. 소스코드는 검증하였고, 이미지나 글의 출처 또한 성실하게 표기하였다. 실수로 누락되었거나 빠뜨린 출처에 대해서는 꼭 보완하여 표기하도록 하겠다. IoT 관련분야나 연관 기술등에 대해 관심있는 사용자는 참고하길 바란다.

[1] (주)휴인스 : http://www.huins.com

[2] (주)휴인스 IoT Blog : smartiot.tistory.com

[3] 라즈베리 파이 활용백서 : 실전 프로젝트 20 - 이재상 표윤석 지음, 비제이 퍼블릭

[4] 라즈베리 파이 산딸기 마을 카뮤니티 http://www.rasplay.org/

[5] 리눅스와 함께하는 라즈베리 파이 - 피터 멤브리, 배장열 옮김, 제이펍

[6] ITU-T Internet Report "The Internet of Things - Executive Summary", Nov. 2005.

[7] 민경식, Net Term, 인터넷 & 시큐리티 이슈, "사물 인터넷 (Internet of Things)", 2012.

[8] 전자부품연구원(KETI), "IoT(Internet of Things) Presentation Design", Nov. 2012.

[9] 이준섭, TTA 표준화 교육, 스마트 통신시장의 핵심열쇠 "M2M", "M2M/IoT Standardization in ITU-T", Oct. 23, 2012.

[10] ITU-T, ITU-T SG13, "Y.WoT, Framework of Web of Things", Oct. 10, 2011.

[11] ITU-T, ITU-T SG13, "Y.2060, Overview of the Internet of Things", July 04, 2012.

[12] IETF, Draft-lee-iot-problem-statement-05.txt

[13] JP Vasseur, IETF 77, Internet Area Meeting, "The Internet of Things", March 2010.

[14] Internet of Things - Feb 6.2013, Yim Hyung-jun, Database Systems Lab. Dept. of Computer Engineering Chungnam National University

[15] 사물인터넷(IoT)분야의 주요 임베디드 SW 기술 - MDS 테크놀로지㈜ 장성균(2014/07/24)

[16] 사물인터넷 글로벌 생태계 구축 및 서비스 활성화를 위한 개방형 IoT 소프트웨어 플랫폼 기술 - 2013. 10. 25, 김재호 (jhkim@keti.re.kr), 전자부품연구원

[17] "라즈베리파이 공식 사이트", http://www.raspberrypi.org/

[18] "element14 의 레즈베리파이 커뮤니티", http://www.element14.com/community/groups/raspberry-pi

[19] "RASPBERRY PI INFORMATION AND VIDEOS", http://raspi.tv/

[20] "Raspberry Pi Spy", http://www.raspberrypi-spy.co.uk/

[21] "RaspiHub", http://www.raspihub.com/

[22] "Adafruit Blog", http://www.adafruit.com/blog/category/raspberry-pi/

[23] "Raspberry Pi Pod", http://www.recantha.co.uk/blog/

[24] "bcm2835", http://www.open.com.au/mikem/bcm2835/

[25] "Gordons Projects", https://projects.drogon.net/raspberry-pi/wiringpi/

[26]http://www.sainsmart.com/sainsmart-mifare-rc522-card-read-antenna-rf-rfid-reader-ic-card-proximity-module.html

[27]https://github.com/lthiery/SPI-Py

[28]https://github.com/mxgxw/MFRC522-python